

Unit -II

Divide and Conquer Technique

General Method:

- The Divide and Conquer Technique splits n inputs into k subsets, $1 < k \leq n$, resulting k subproblems.
- These subproblems will be solved and then combined by using a separate method to get a solution to the whole problem.
- If the subproblems are large, then the Divide and Conquer Technique will be reapplied.
- Generally, subproblems coming from a Divide and Conquer Technique are of the same type as the original problem.

- The reapplication of the Divide and Conquer Technique is naturally expressed by a recursive algorithm.
- Now smaller and smaller problems of the same kind are generated until subproblems that are small enough to solve without splitting further.

Divide

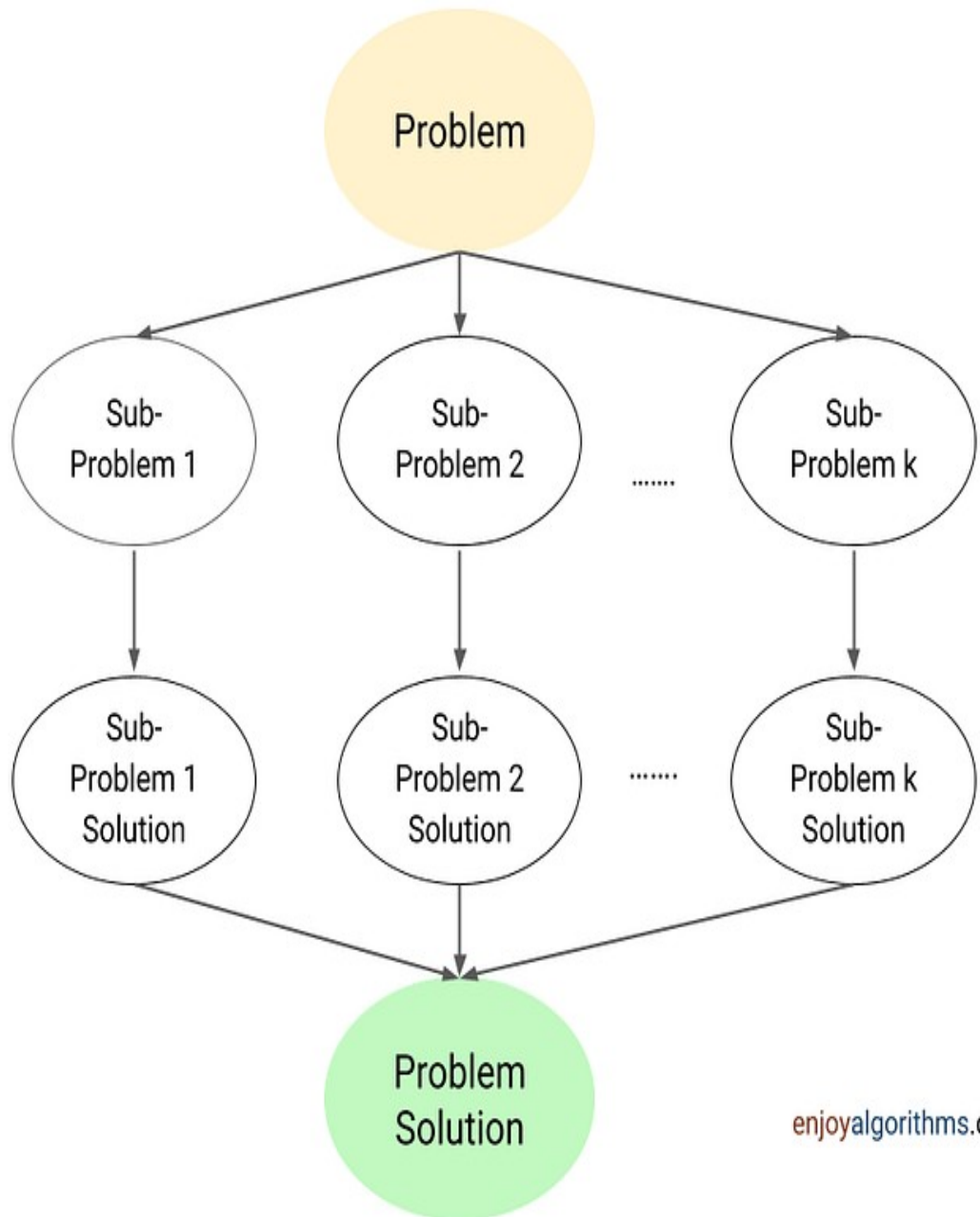
Dividing the problem into smaller sub-problems

Conquer

Solving each sub-problems recursively

Combine

Combining sub-problem solutions to build the original problem solution



Control Abstraction / General Method for Divide and Conquer Technique

Algorithm DAndC(p)

```
{  
  if Small(p) then return s(p);  
  else  
  {  
    divide p into smaller problems p1,p2,.....,pk,  $k \geq 1$ ;  
    Apply DAndC to each of these subproblems;  
    return Combine(DAndC(p1),          DAndC(p2),  
    .....,DAndC(pk));  
  }  
}
```

- The Time Complexity of many divide-and-conquer algorithms is given by recurrences of the form

$$T(n) = \begin{cases} T(1) & \text{when } n=1 \\ bT(n/b) + f(n) & \text{when } n>1 \end{cases}$$

- Where $T(n)$ is the time for DAndC on any input of size n
- The function $f(n)$ is the time for dividing the problem and combining the solutions of subproblems.
- Where a, b are known constants, we assume that $T(1)$ is known and n is a power of b (i.e., $n = b^k$)

Applications

1. Binary search Algorithm

Iterative Method

Algorithm BinSearch(a, n, x)

// a is an array of size n, x is the key element to be searched.

```
{    low:=1;  high:=n;
  while( low ≤ high)
  {
      mid:=(low+high)/2;

      if( x < a[mid] ) then high := mid-1;

      else if( x > a[mid] ) then low := mid+1;

      else  return mid;
  }
  return 0;
}
```

Recursive Algorithm (Divide and Conquer Technique)

Algorithm BinSrch (a, low, high, x)

//Given an array a [low : high] of elements in increasing

//order, $1 \leq \text{low} \leq \text{high}$, determine whether x is present, and

//if so, return j such that $x = a[j]$; else return 0.

{

if(low = high) **then** // If small(P)

 {

if($x = a[\text{low}]$) **then return** low;

else return 0;

 }

else


```
{  
    //Reduce p into a smaller subproblem.  
    mid:= (low+high)/2  
    if( x = a[mid] ) then return mid;  
    else if ( x<a[mid] ) then  
        return BinSrch(a, low, mid-1, x);  
    else  
        return BinSrch(a, mid+1, high, x);  
}  
}
```

Time complexity of Binary Search

- If the time for dividing the list is a constant, then the computing time for binary search is described by the recurrence relation

$$T(n) = \begin{cases} c_1 & n=1, c_1 \text{ is a constant} \\ T(n/2) + c_2 & n>1, c_2 \text{ is a constant} \end{cases}$$

Assume $n = 2^k$, then

$$\begin{aligned} T(n) &= T(n/2) + c_2 \\ &= T(n/4) + c_2 + c_2 \\ &= T(n/8) + c_2 + c_2 + c_2 \\ &\dots \\ &\dots \\ &= T(n / 2^k) + c_2 + c_2 + c_2 + \dots \dots \dots k \text{ times} \\ &= T(1) + kc_2 \\ &= c_1 + kc_2 = c_1 + \log n * c_2 = O(\log n) \end{aligned}$$

Time Complexity of Binary Search

Successful searches:

best	average	worst
$O(1)$	$O(\log n)$	$O(\log n)$

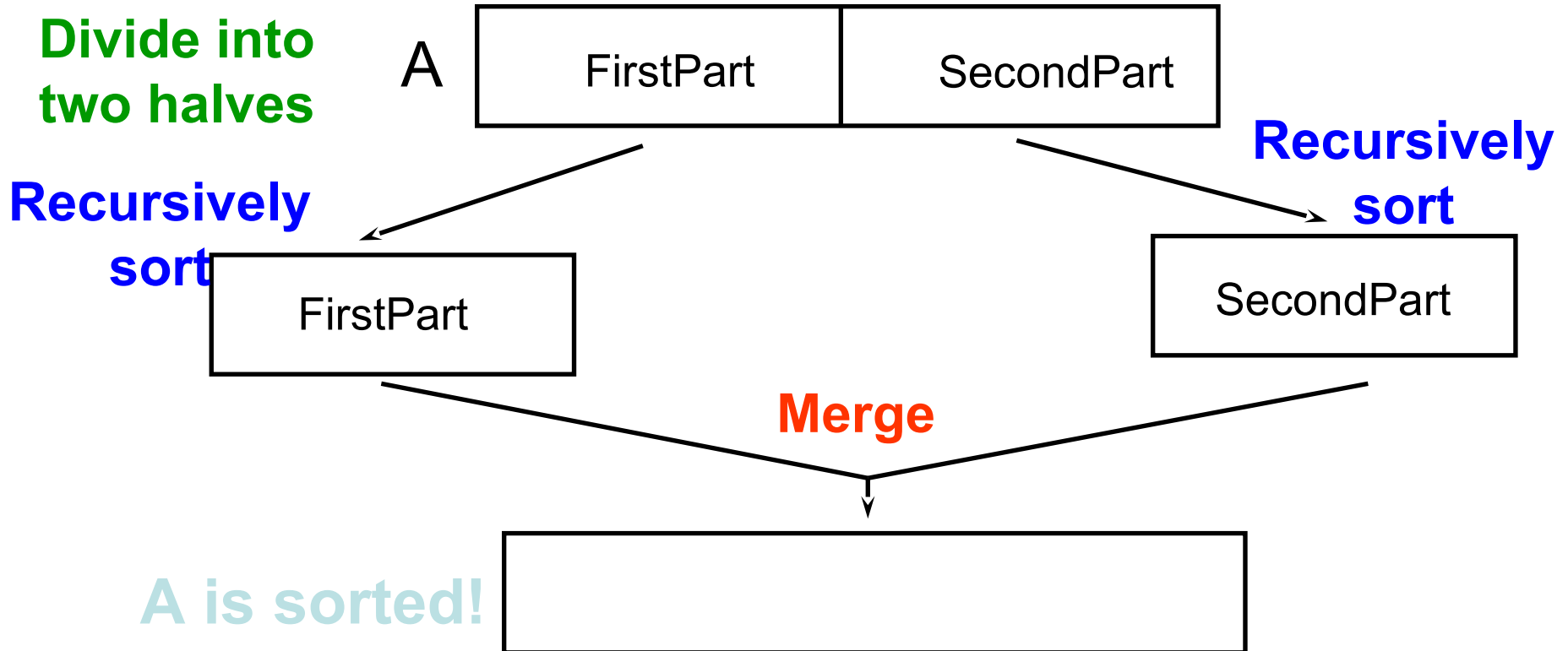
Unsuccessful search:

$O(\log n)$

2. Merge Sort

1. **Base Case**, solve the problem **directly** if it is small enough(**only one element**).
2. **Divide** the problem into two or more **similar** and **smaller** subproblems.
3. **Recursively** solve the subproblems.
4. **Combine** solutions to the subproblems.

Merge Sort: Idea



Merge-Sort(A, 0, 7)

Divide

A:



Merge-Sort(A, 0, 7)

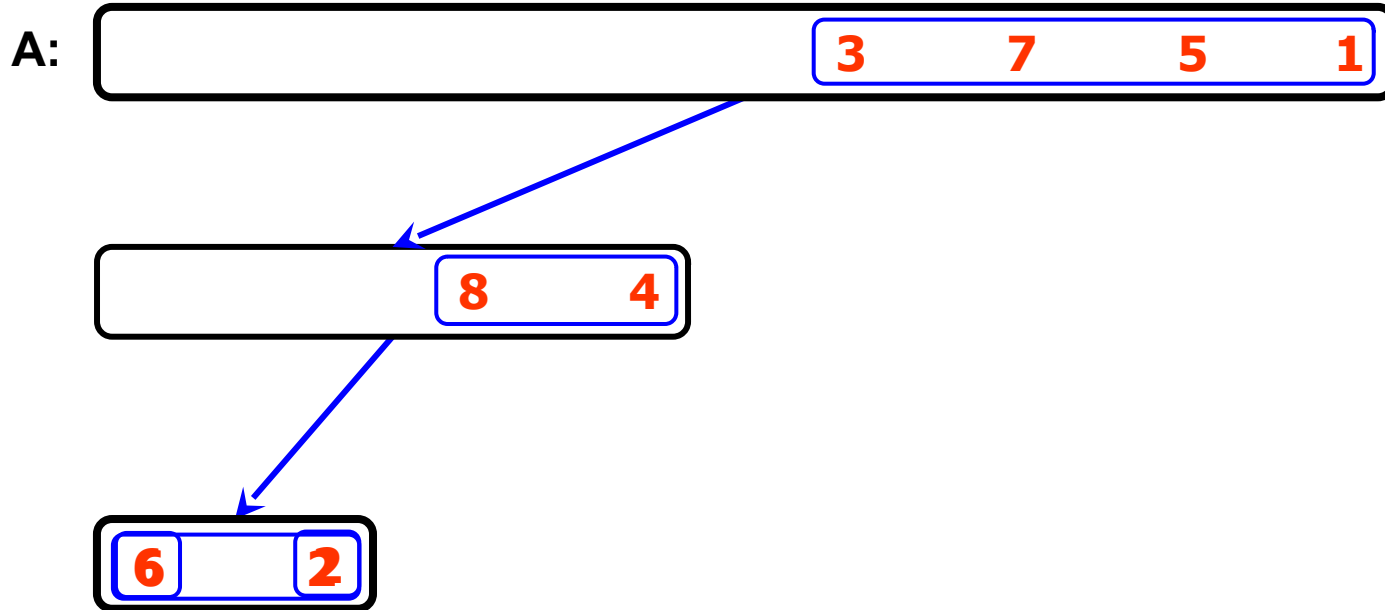
Merge-Sort(A, 0, 3), divide

A:



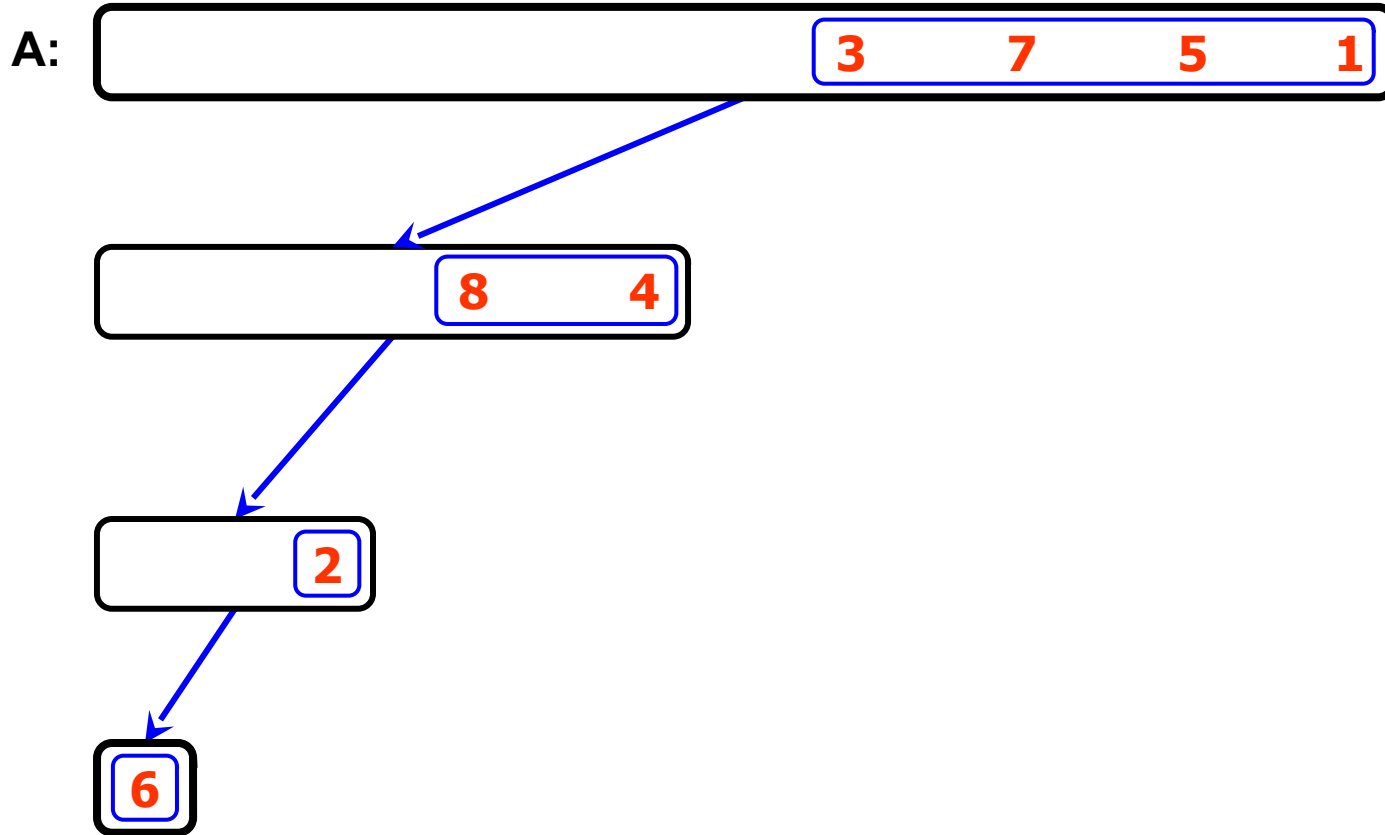
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 1), divide



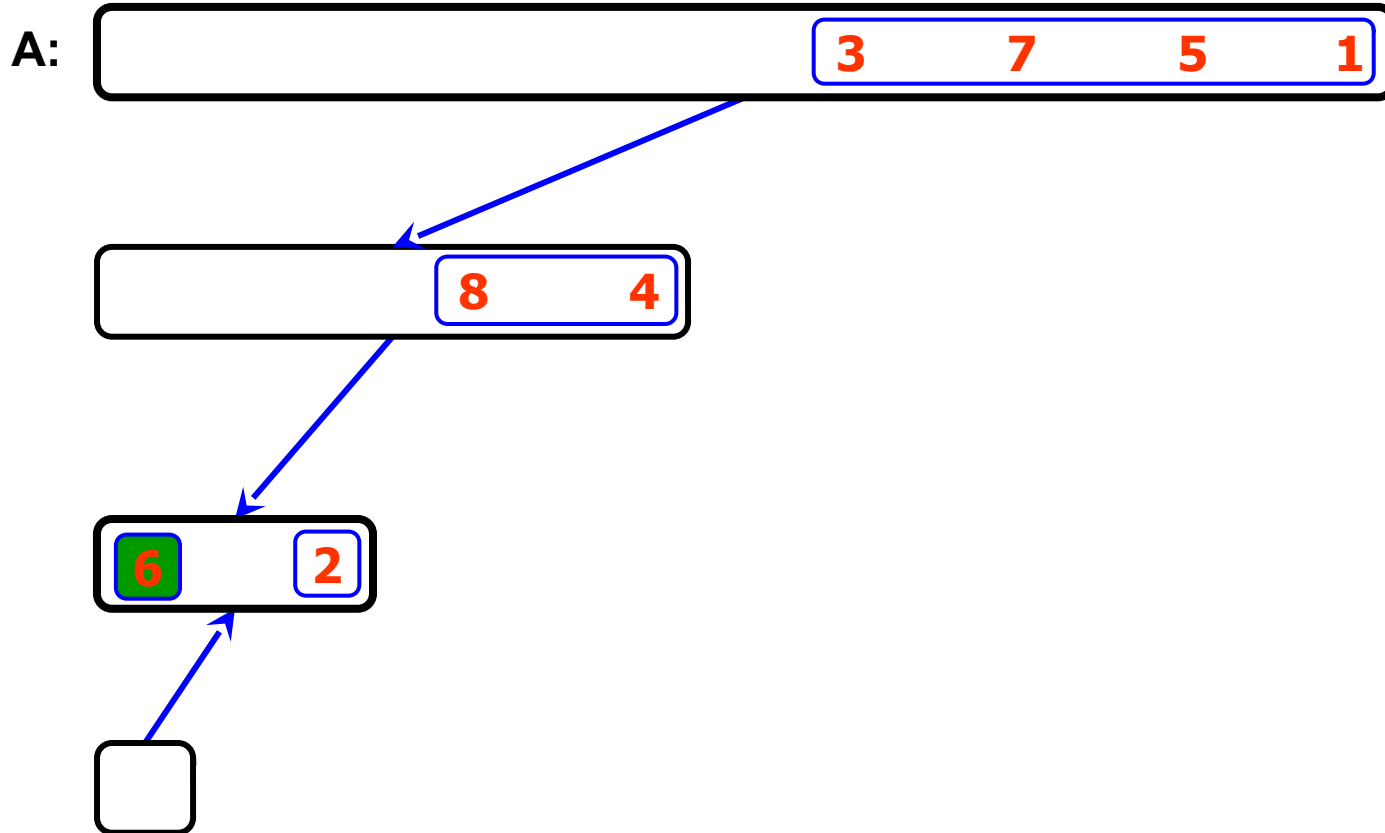
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 0) , base case



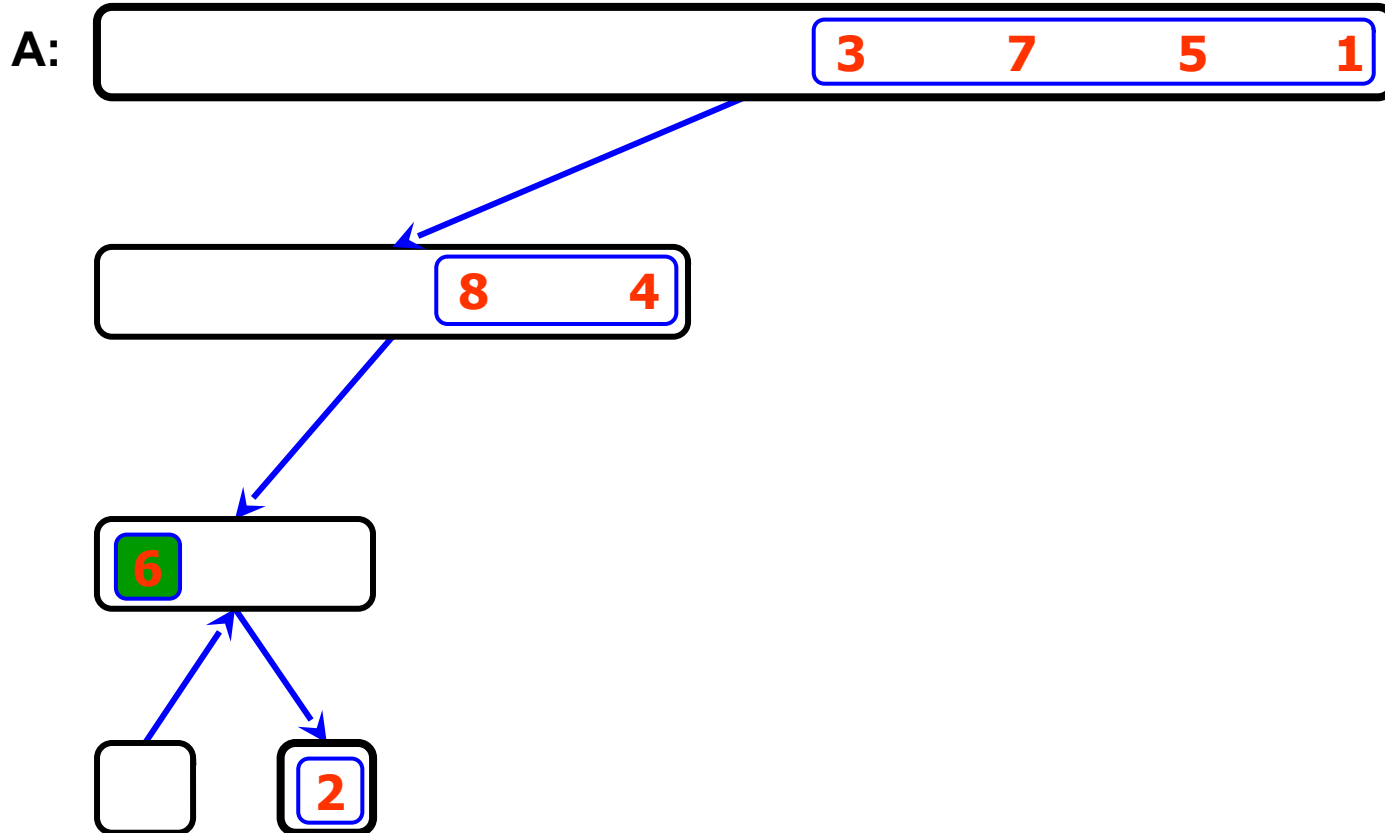
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 0), return



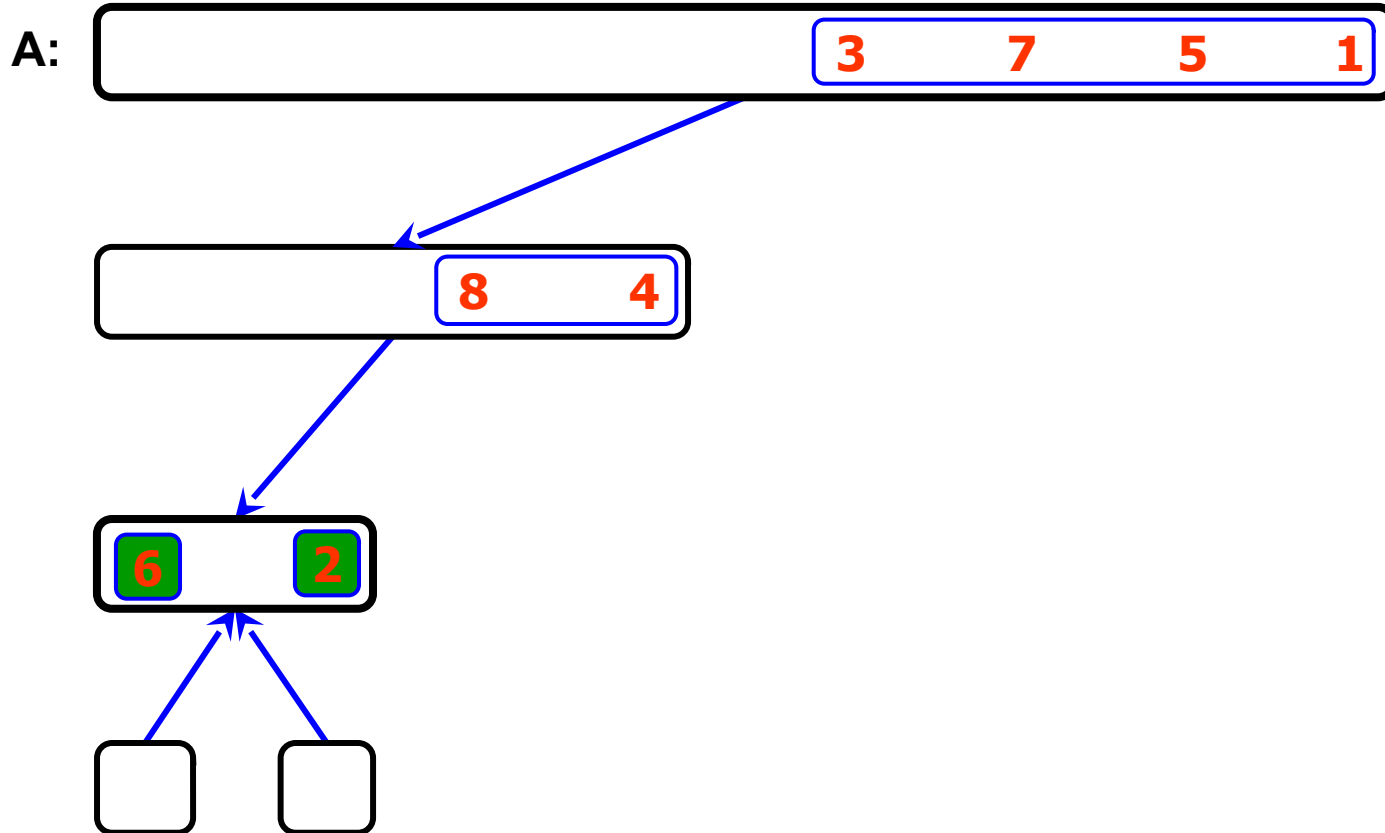
Merge-Sort(A, 0, 7)

Merge-Sort(A, 1, 1), base case



Merge-Sort(A, 0, 7)

Merge-Sort(A, 1, 1), return



Merge-Sort(A, 0, 7)

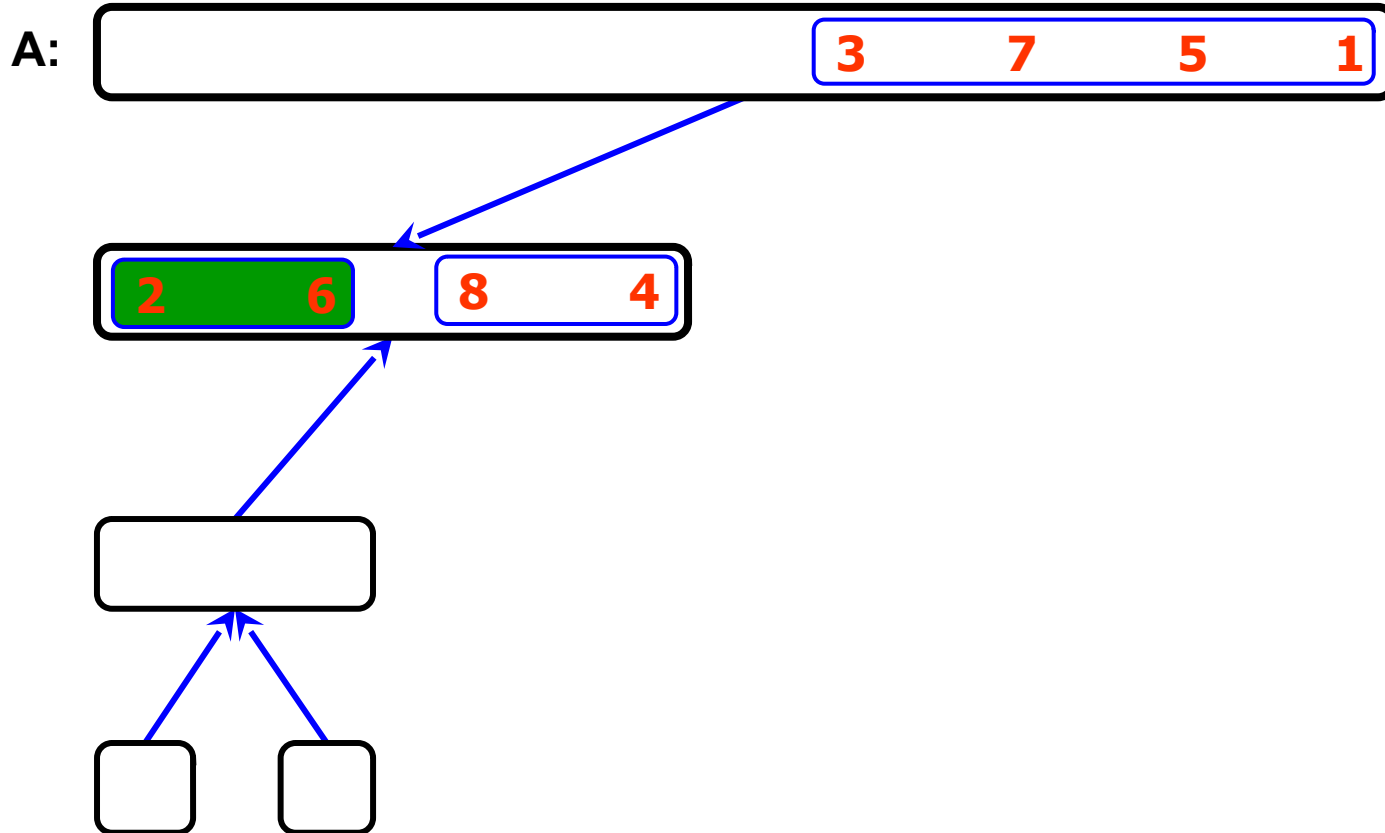
Merge(A, 0, 0, 1)

A:



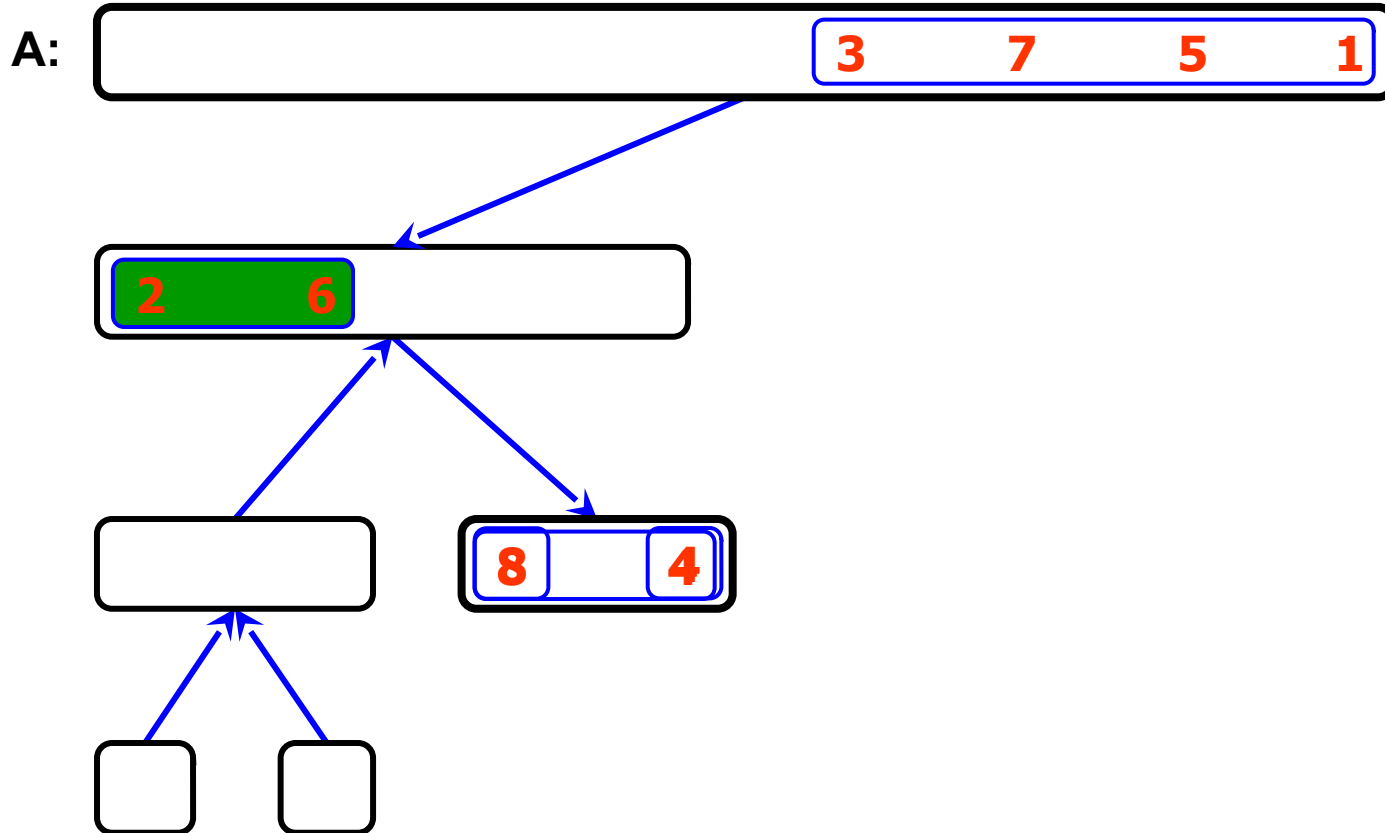
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 1), return



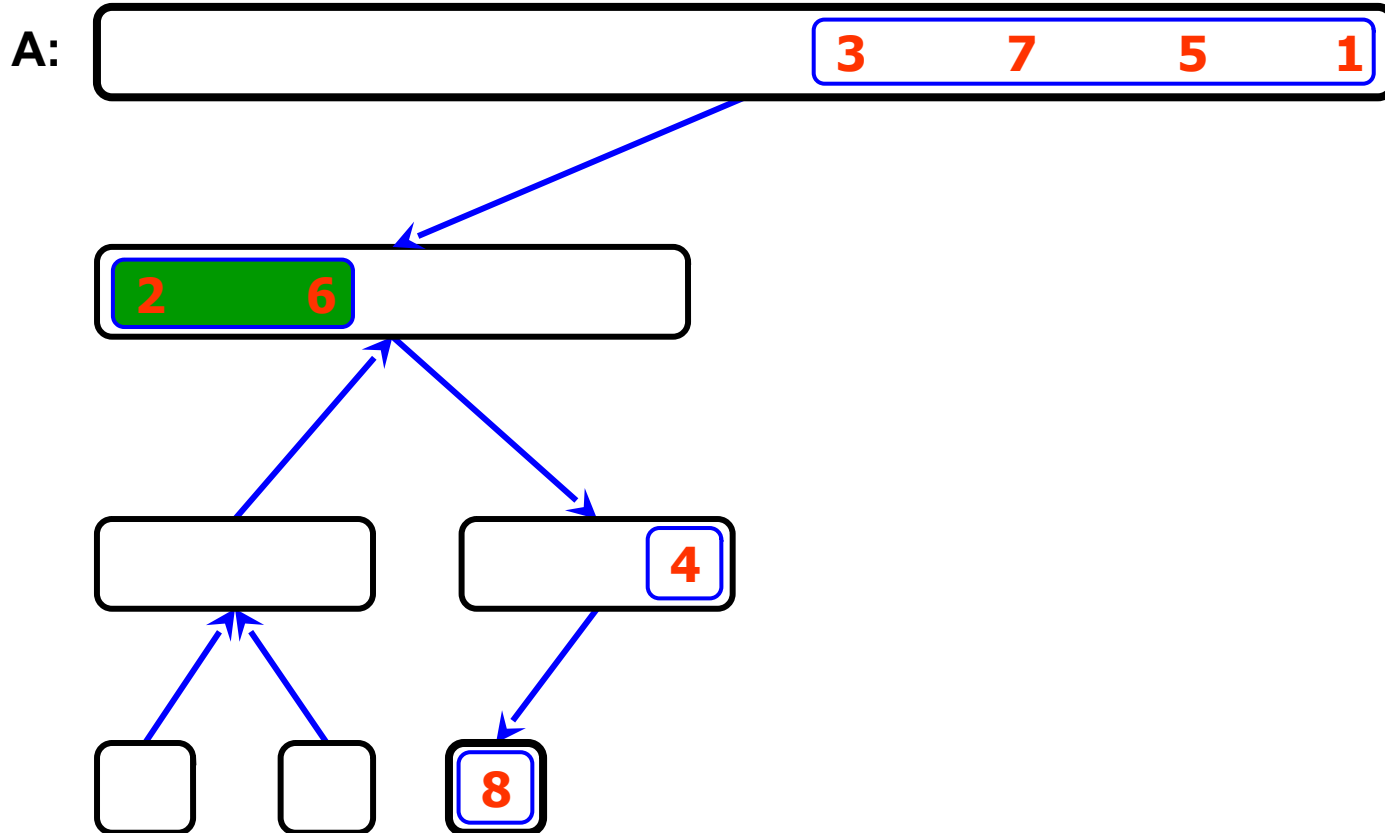
Merge-Sort(A, 0, 7)

Merge-Sort(A, 2, 3), divide



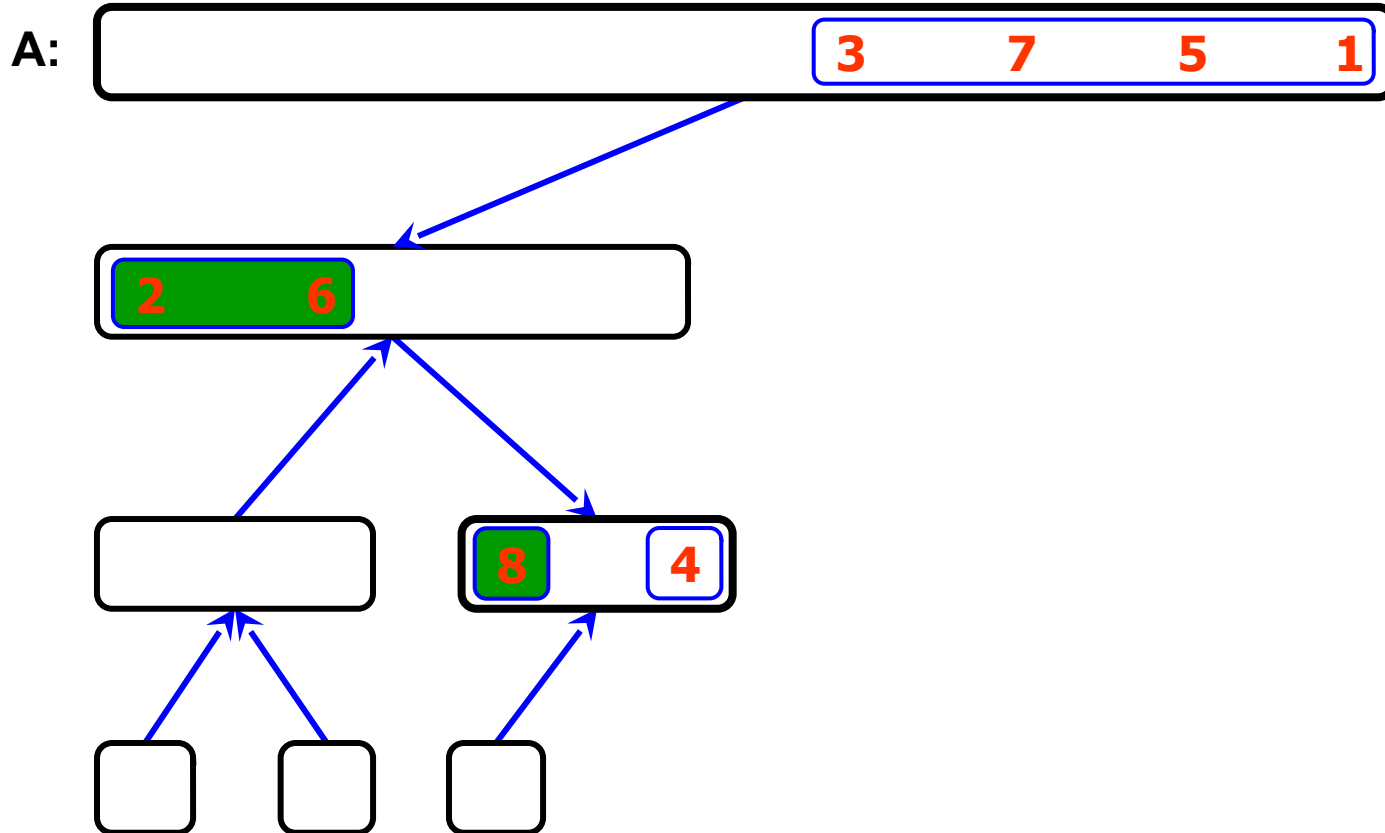
Merge-Sort(A, 0, 7)

Merge-Sort(A, 2, 2), base case



Merge-Sort(A, 0, 7)

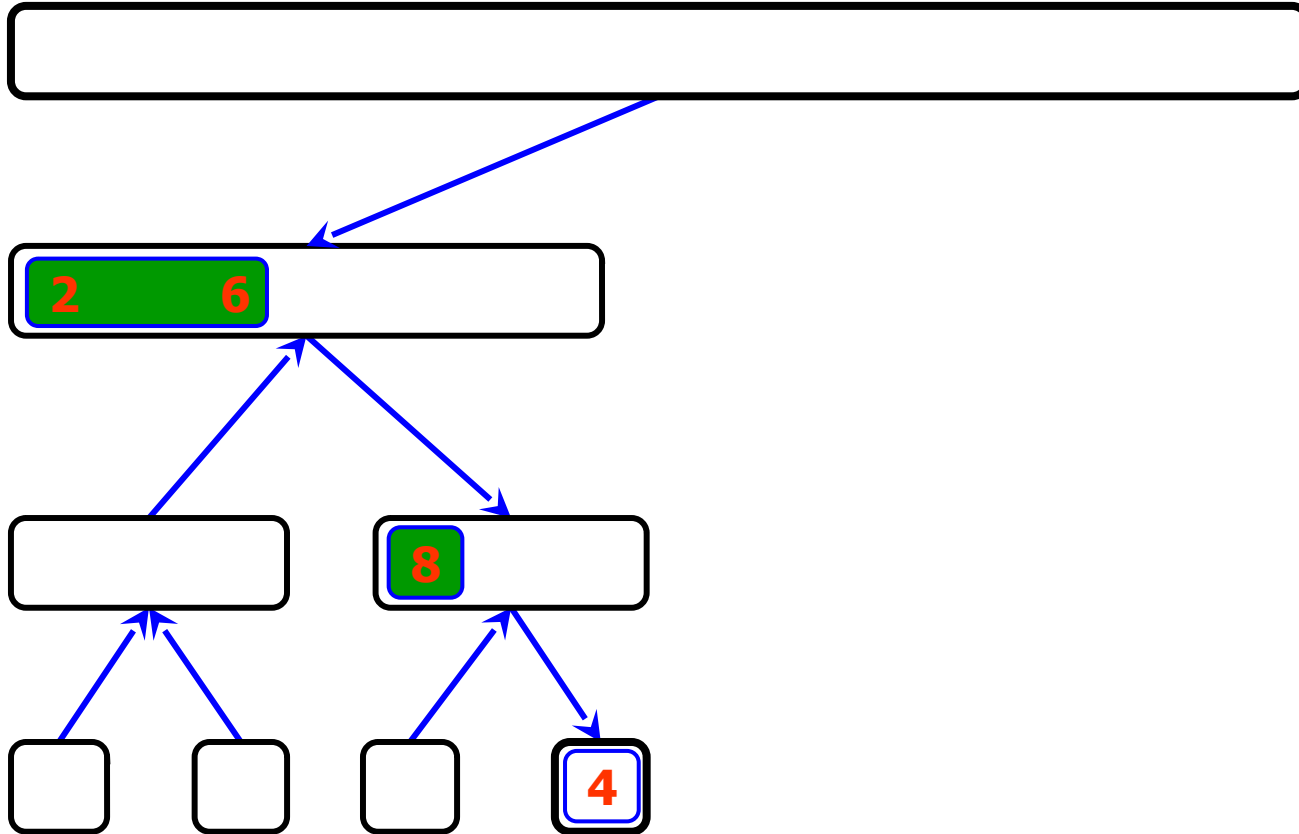
Merge-Sort(A, 2, 2), return



Merge-Sort(A, 0, 7)

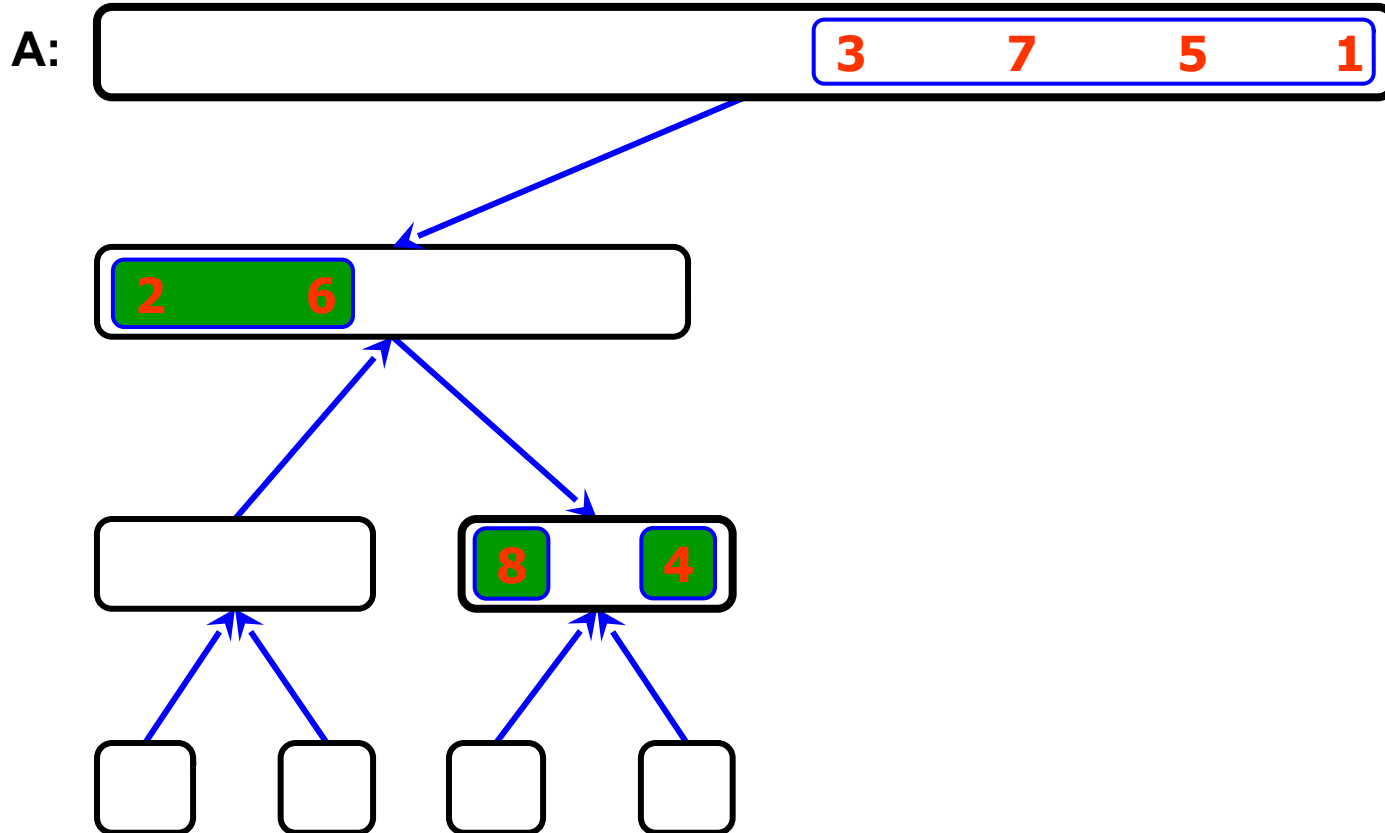
Merge-Sort(A, 3, 3), base case

A:



Merge-Sort(A, 0, 7)

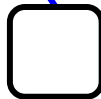
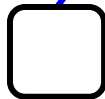
Merge-Sort(A, 3, 3), return



Merge-Sort(A, 0, 7)

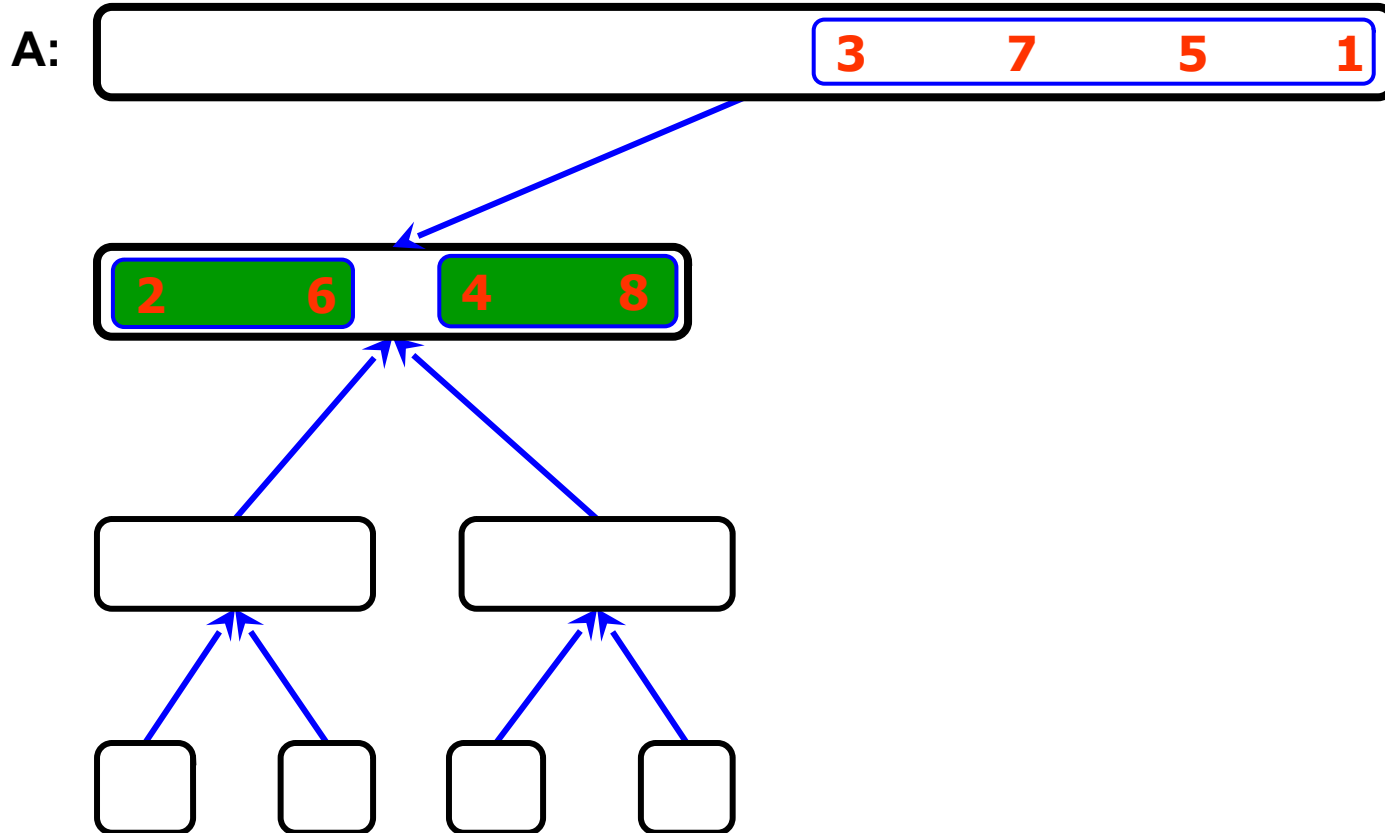
Merge(A, 2, 2, 3)

A:



Merge-Sort(A, 0, 7)

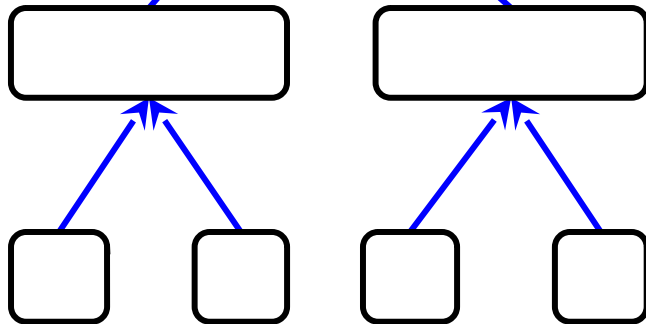
Merge-Sort(A, 2, 3), return



Merge-Sort(A, 0, 7)

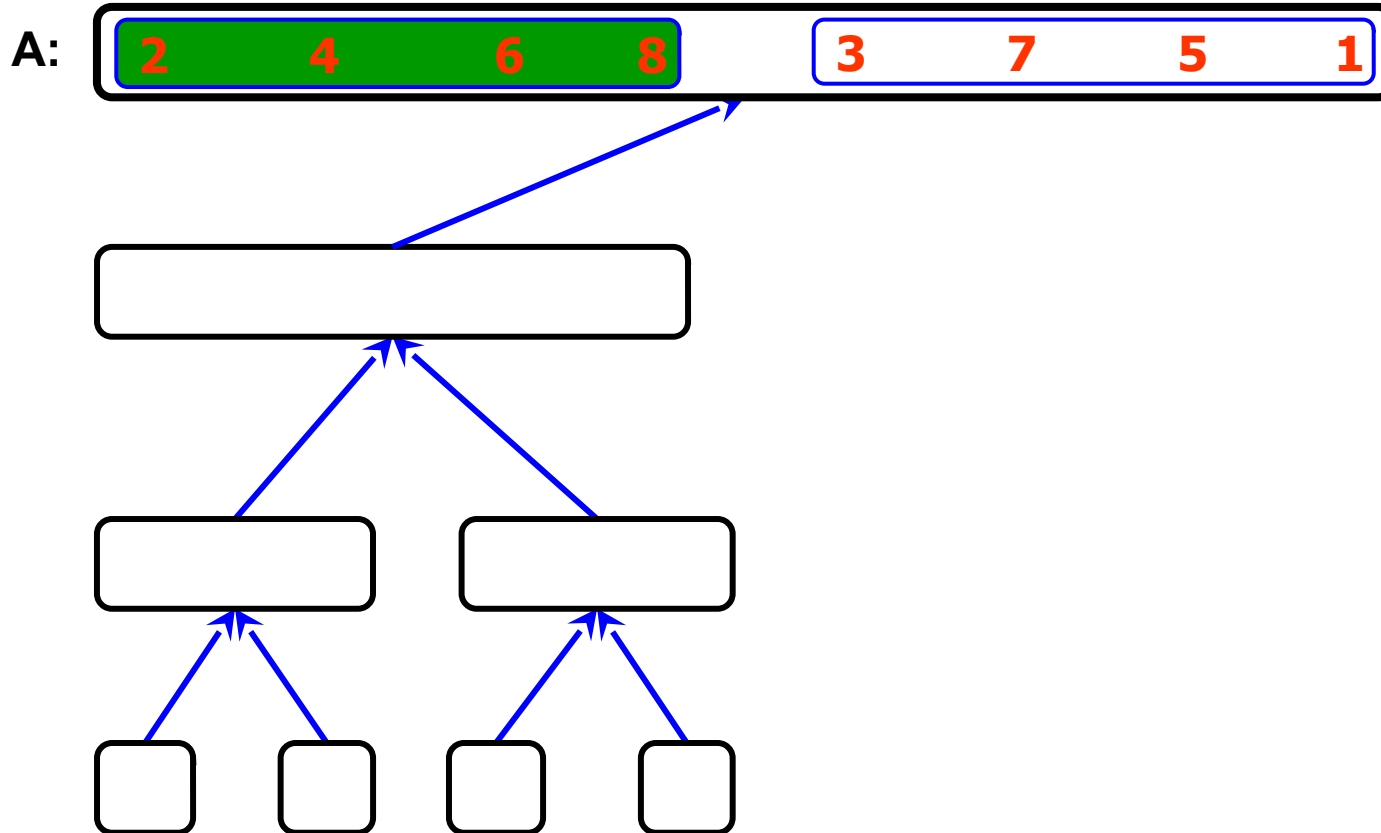
Merge(A, 0, 1, 3)

A:



Merge-Sort(A, 0, 7)

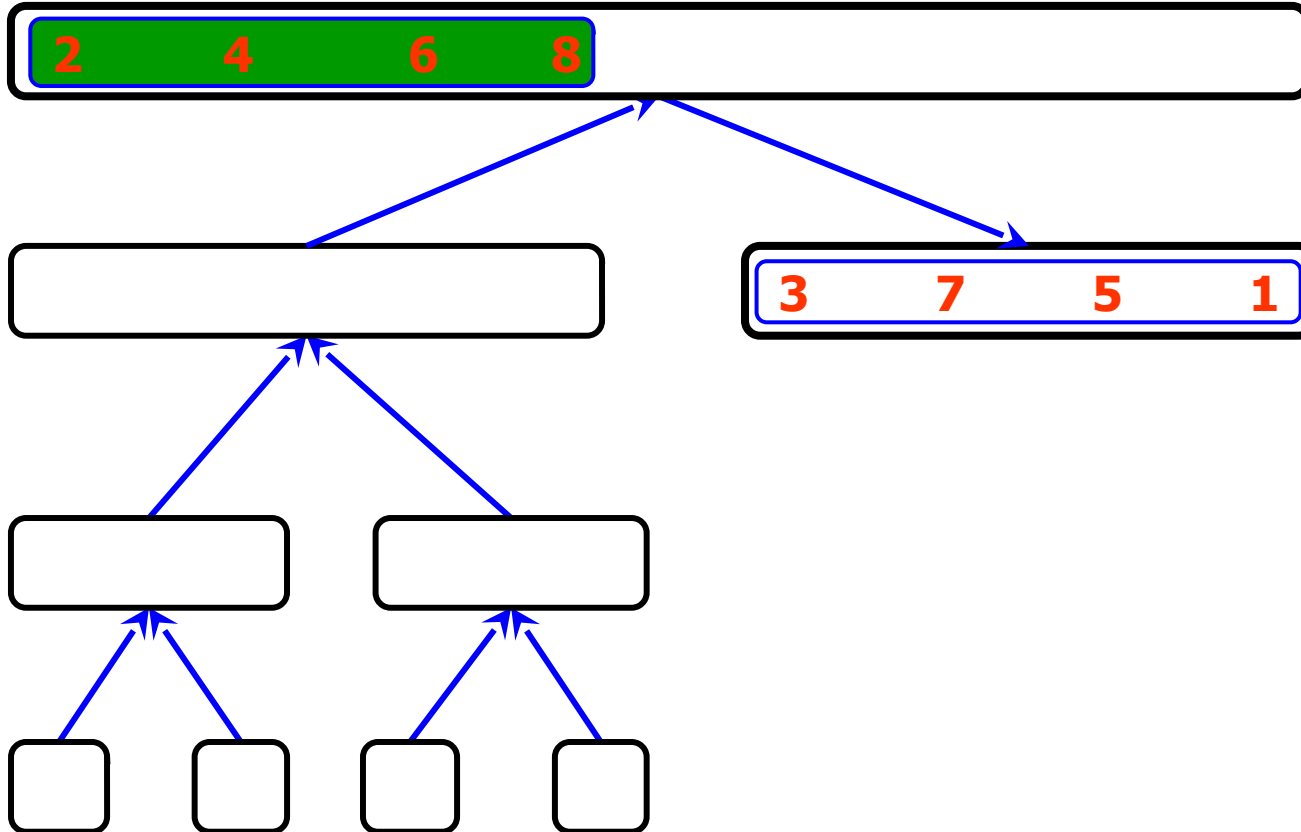
Merge-Sort(A, 0, 3), return



Merge-Sort(A, 0, 7)

Merge-Sort(A, 4, 7)

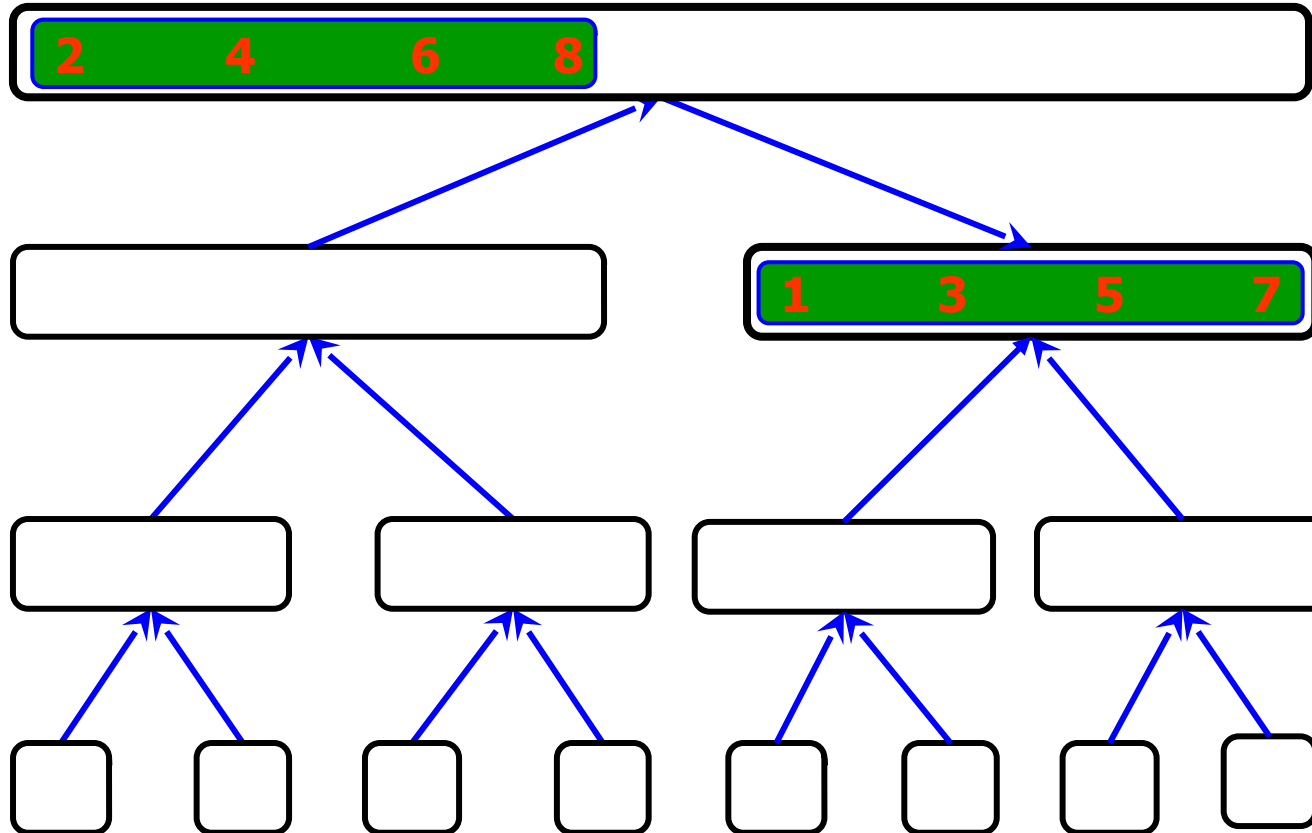
A:



Merge-Sort(A, 0, 7)

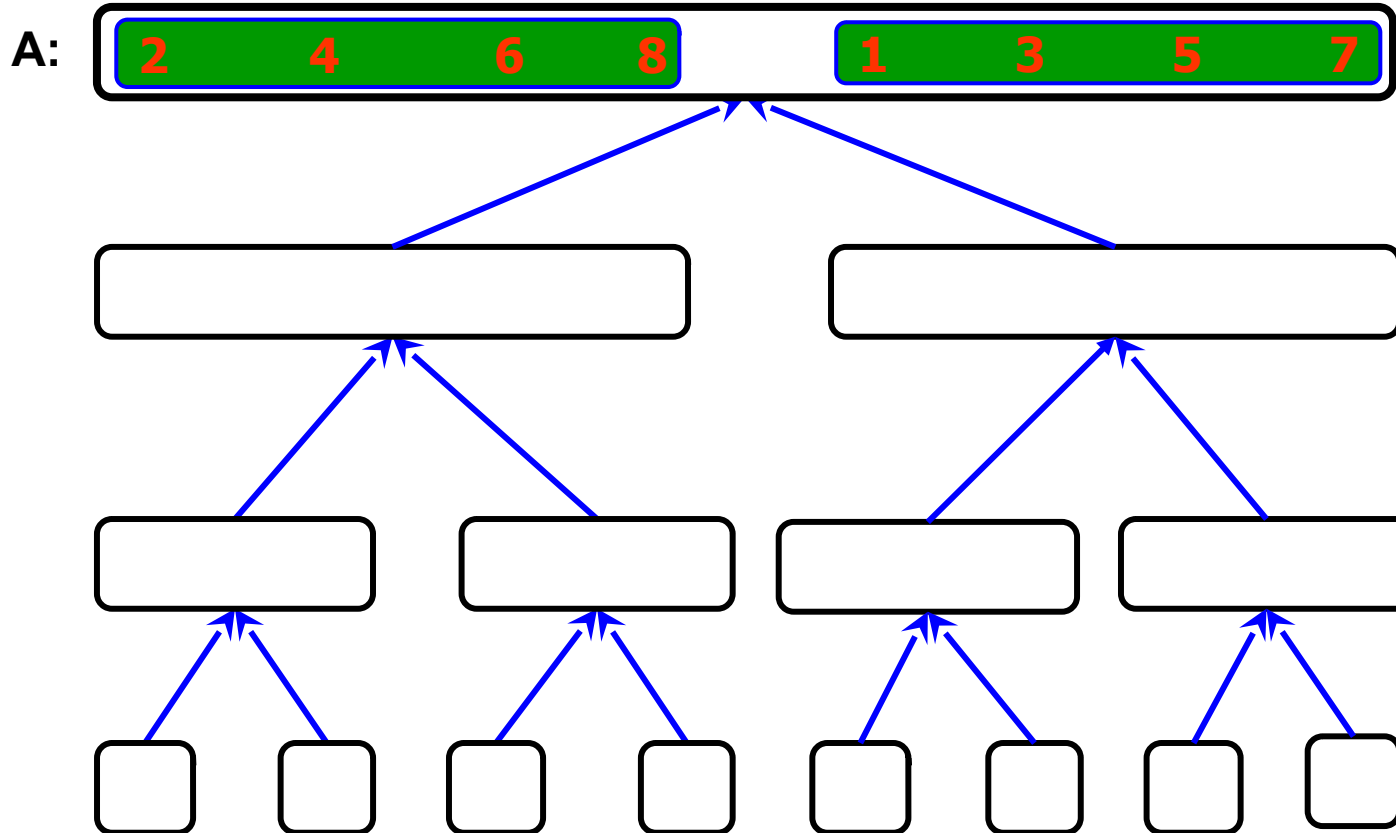
Merge (A, 4, 5, 7)

A:



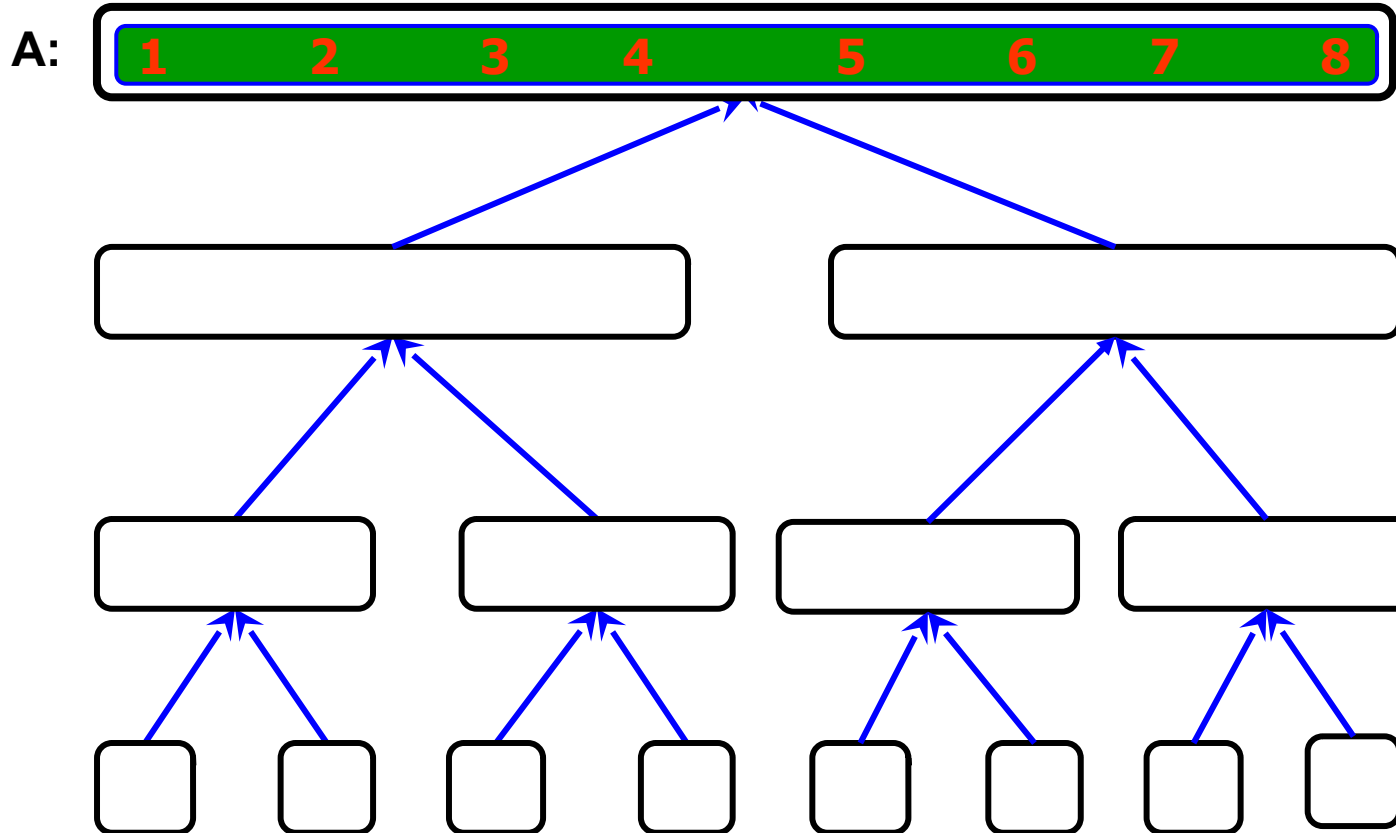
Merge-Sort(A, 0, 7)

Merge-Sort(A, 4, 7), return



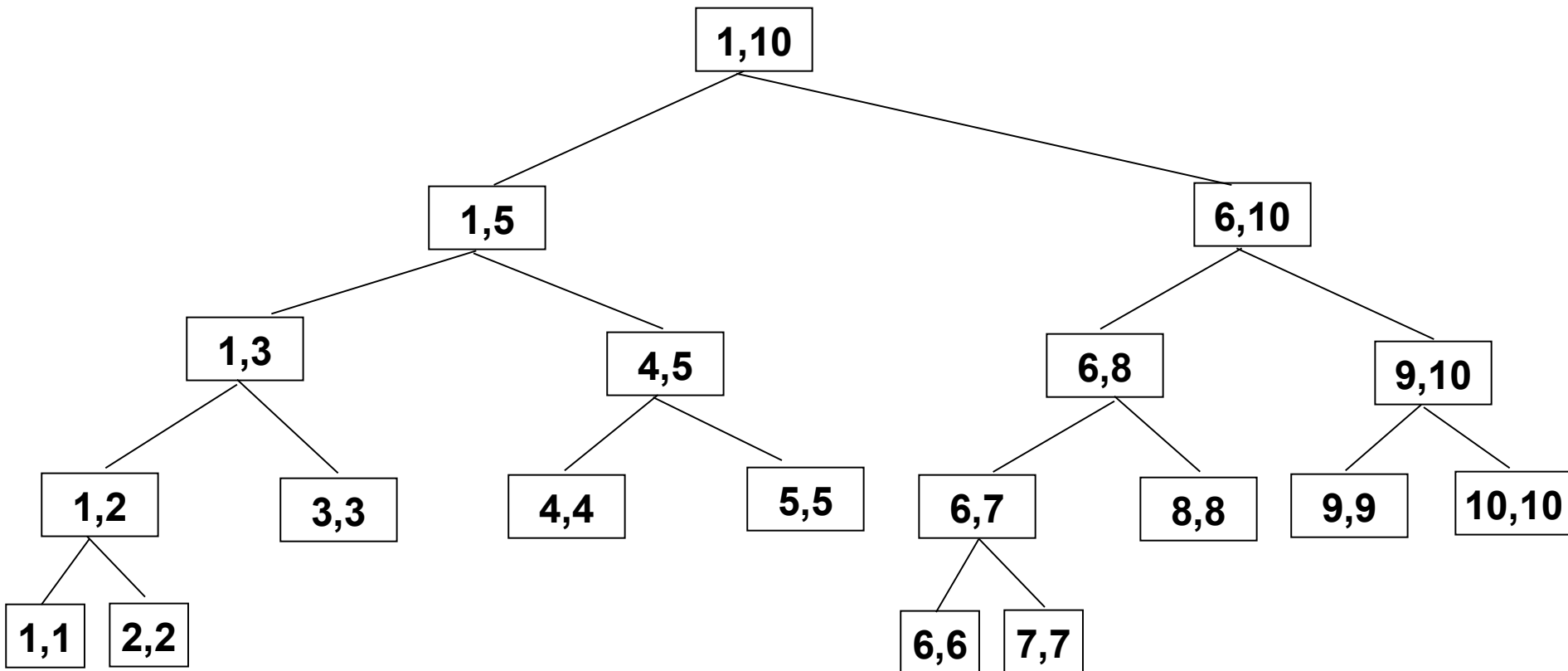
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 7), done!



Ex:- [179, 254, 285, 310, 351, 423, 450, 520,
652,861]

Tree of calls of merge sort



Merge Sort: Algorithm

MergeSort (low,high)

// sorts the elements $a[\text{low}], \dots, a[\text{high}]$ which are in the global array

// $a[1:n]$ into ascending order (increasing order).

// Small(p) is true if there is only one element to sort. In this case the list is

// already sorted.

{ **if** (**low < high**) **then** // if there are more than one element

{

mid $\leftarrow$ (low+high)/2;

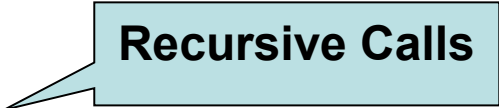
MergeSort(low,mid);

MergeSort(mid+1, high);

Merge(low, mid, high);

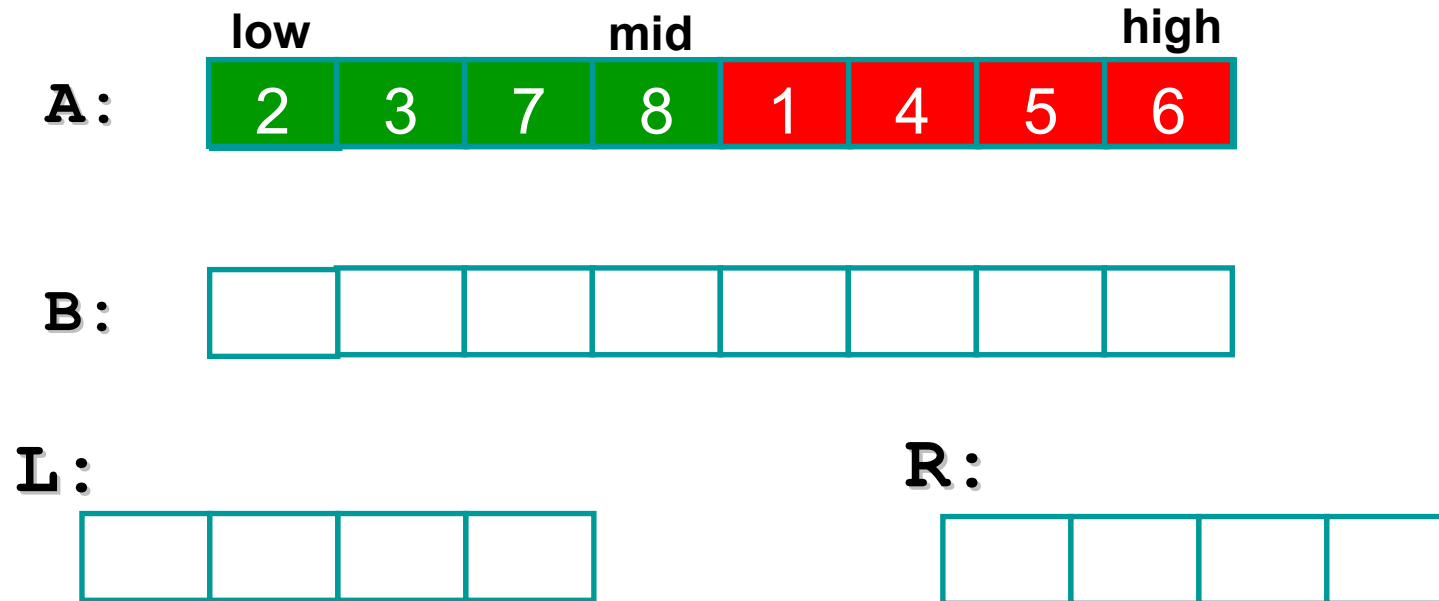
}

}



Recursive Calls

Merge-Sort: Merge Example



Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1							
---	--	--	--	--	--	--	--


 $i = \text{low}$

L:

low		mid	
2	3	7	8

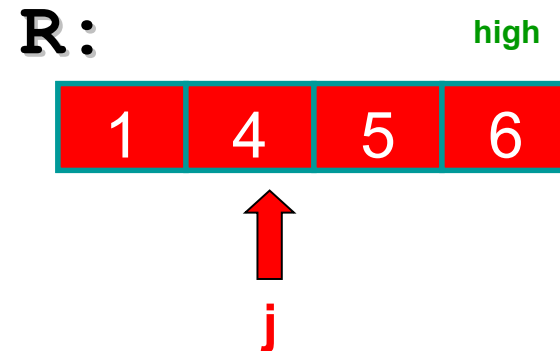
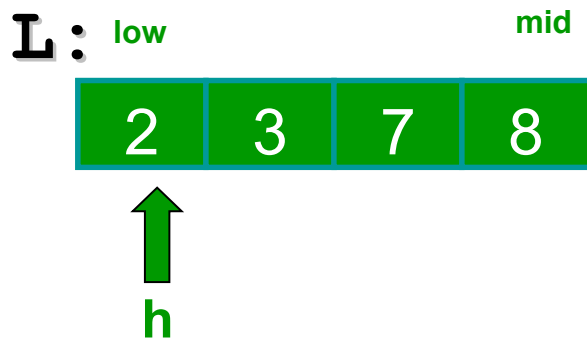
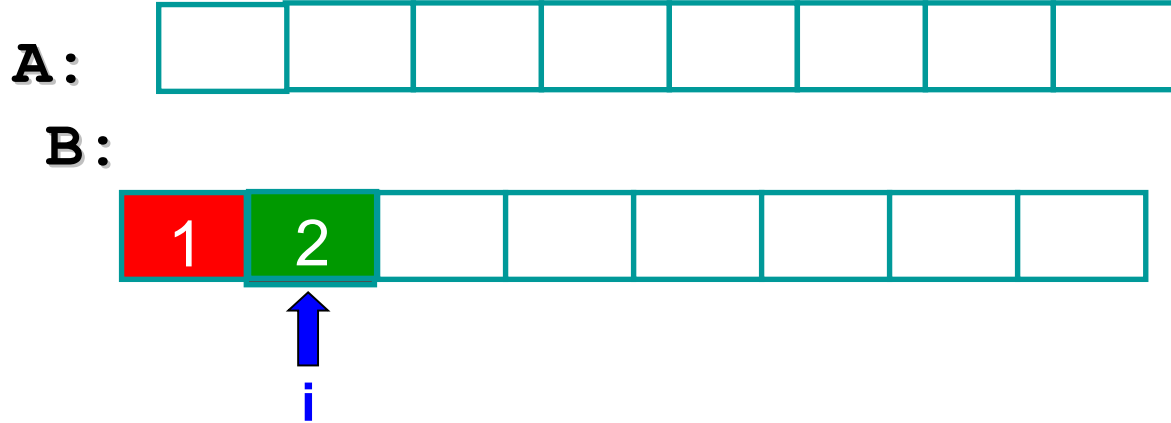

 $h = \text{low}$

R:

high			
1	4	5	6


 $j = \text{mid} + 1$

Merge-Sort: Merge Example

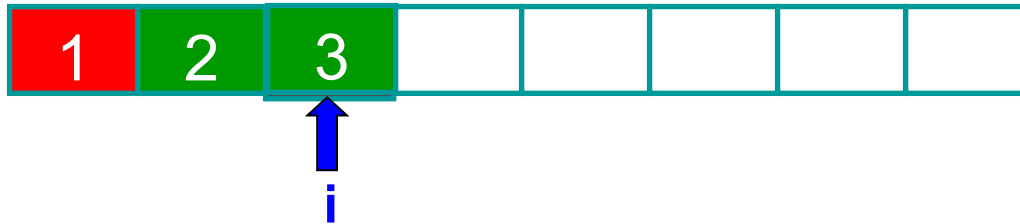


Merge-Sort: Merge Example

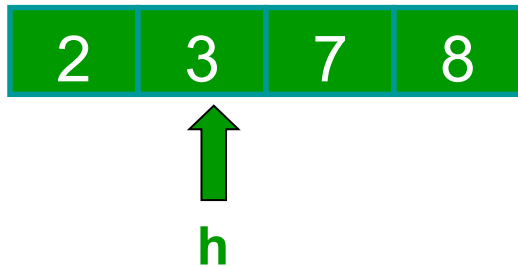
A:

--	--	--	--	--	--	--	--

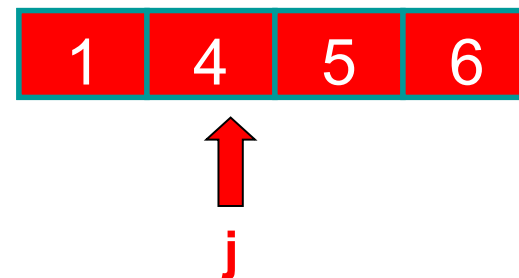
B:



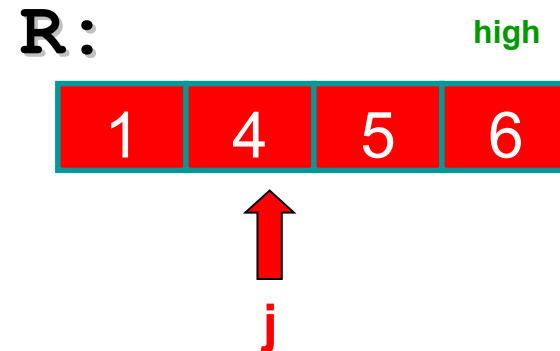
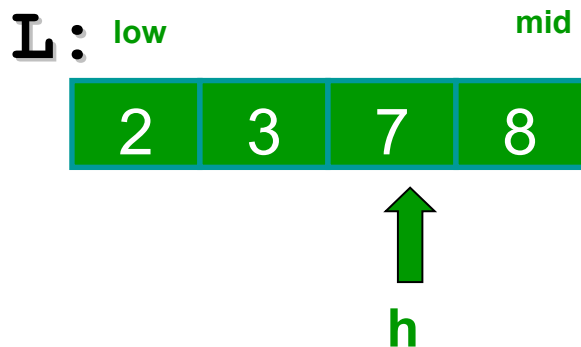
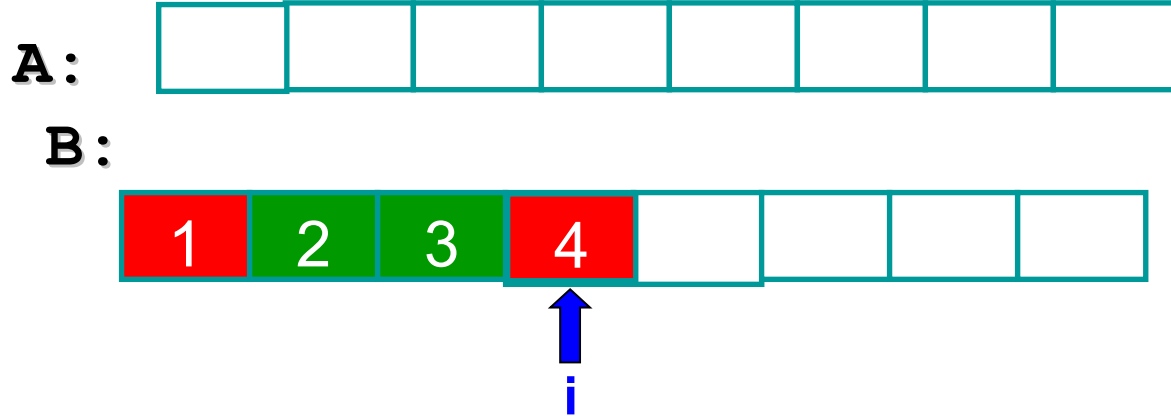
L: low mid



R: high



Merge-Sort: Merge Example

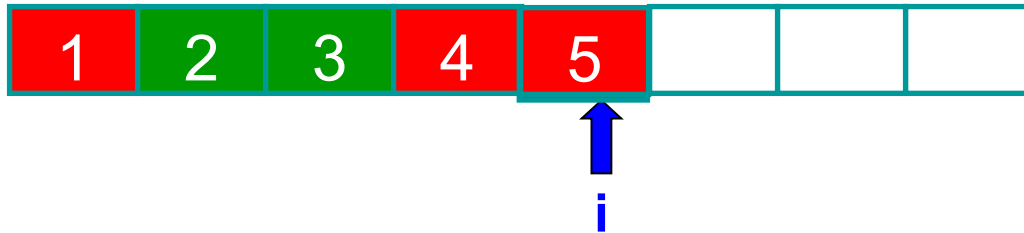


Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:



L: *low*

mid




h

R:

high




j

Merge-Sort: Merge Example

A:



B:



L: low

mid



R:

high



j

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5	6	7	
---	---	---	---	---	---	---	--

↑
i

L: low

mid

2	3	7	8
---	---	---	---

↑
h

R:

high

1	4	5	6
---	---	---	---

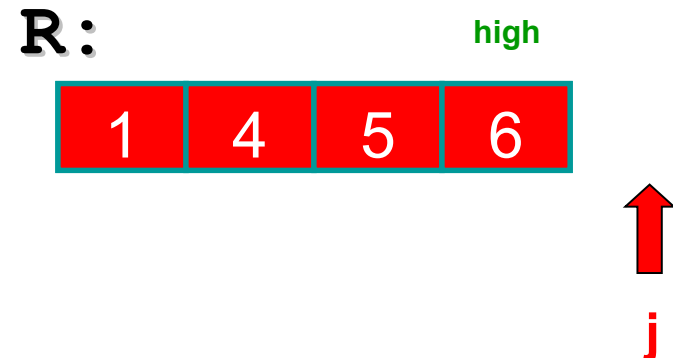
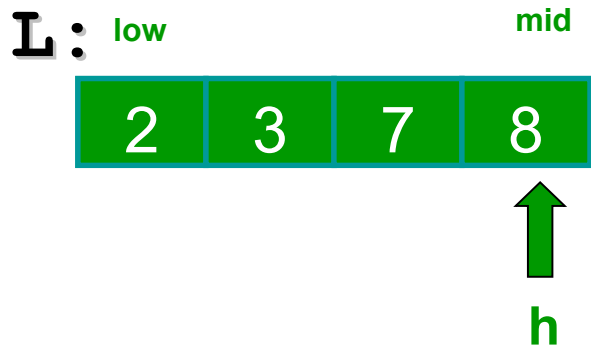
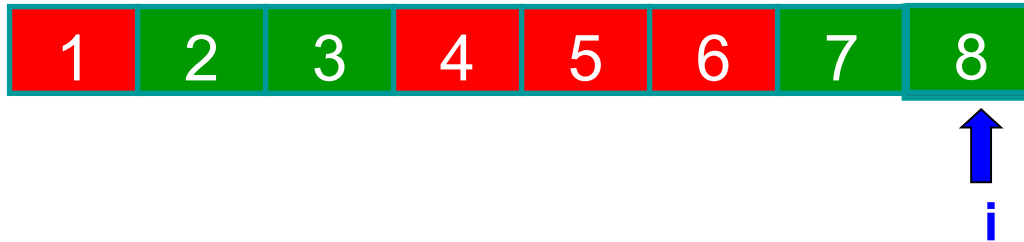
↑
j

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:



Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:



L: low mid



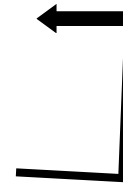
R: high



A:



B:



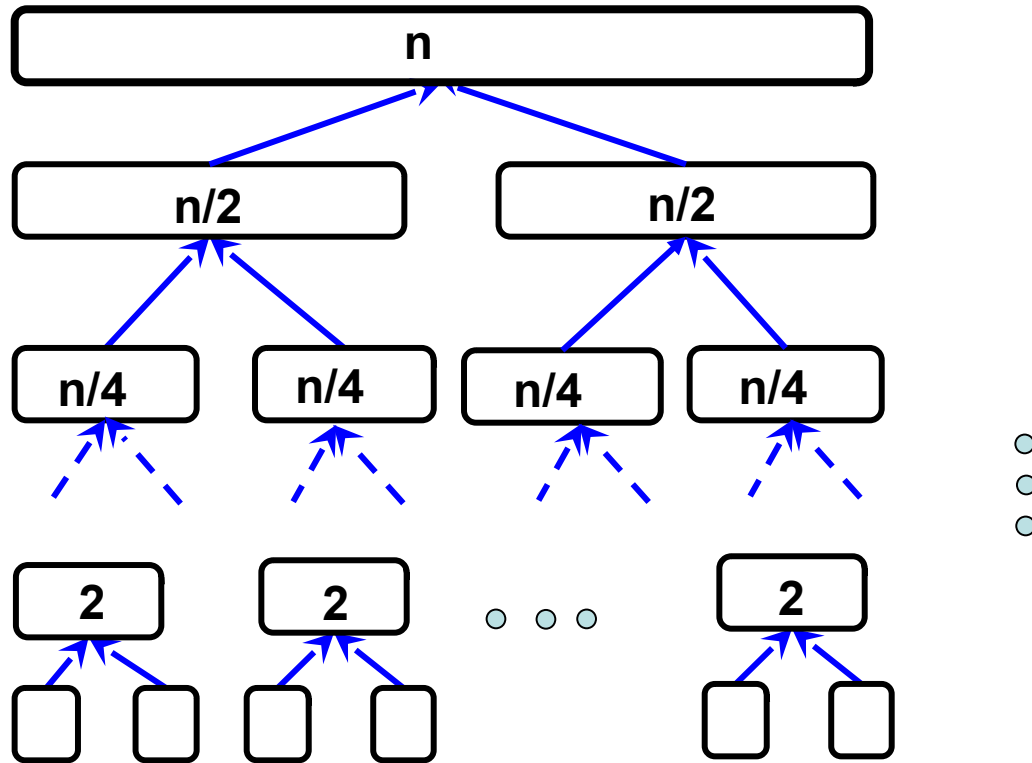
Algorithm Merge(low,mid,high)

// a[low:high] is a global array containing two sorted subsets in a[low:mid]
// and in a[mid+1:high]. The goal is to merge these two sets into a single
// set residing in a [low:high]. b[] is a temporary global array.

```
{  
    h:=low; j:=mid+1; i:=low;  
    while( h ≤ mid ) and ( j ≤ high ) do  
    {  
        if( a[h] ≤ a[j] ) then  
        {  
            b[i]:=a[h]; h:=h+1;  
        }  
        else  
        {  
            b[i]:=a[j]; j:=j+1;  
        }  
        i:=i+1;  
    }  
}
```

```
if( h > mid ) then
    for k:=j to high do
    {
        b[i] := a[k]; i:= i+1;
    }
else
    for k:=h to mid do
    {
        b[i] := a[k]; i:= i+1;
    }
    for k:= low  to high do a[k]:=b[k];
}
```

Merge-Sort Analysis



Merge-Sort Time Complexity

If the time for the merging operation is proportional to n , then the computing time for merge sort is described by the recurrence relation

$$T(n) = \begin{cases} c_1 & n=1, c_1 \text{ is a constant} \\ 2T(n/2) + c_2n & n>1, c_2 \text{ is a constant} \end{cases}$$

Assume $n=2^k$, then

$$\begin{aligned} T(n) &= 2T(n/2) + c_2n \\ &= 2(2T(n/4) + c_2n/2) + c_2n \\ &= 4T(n/4) + 2c_2n \end{aligned}$$

.....

.....

$$= 2^k T(1) + kc_2n$$

$$= c_1n + c_2n \log n = O(n \log n)$$

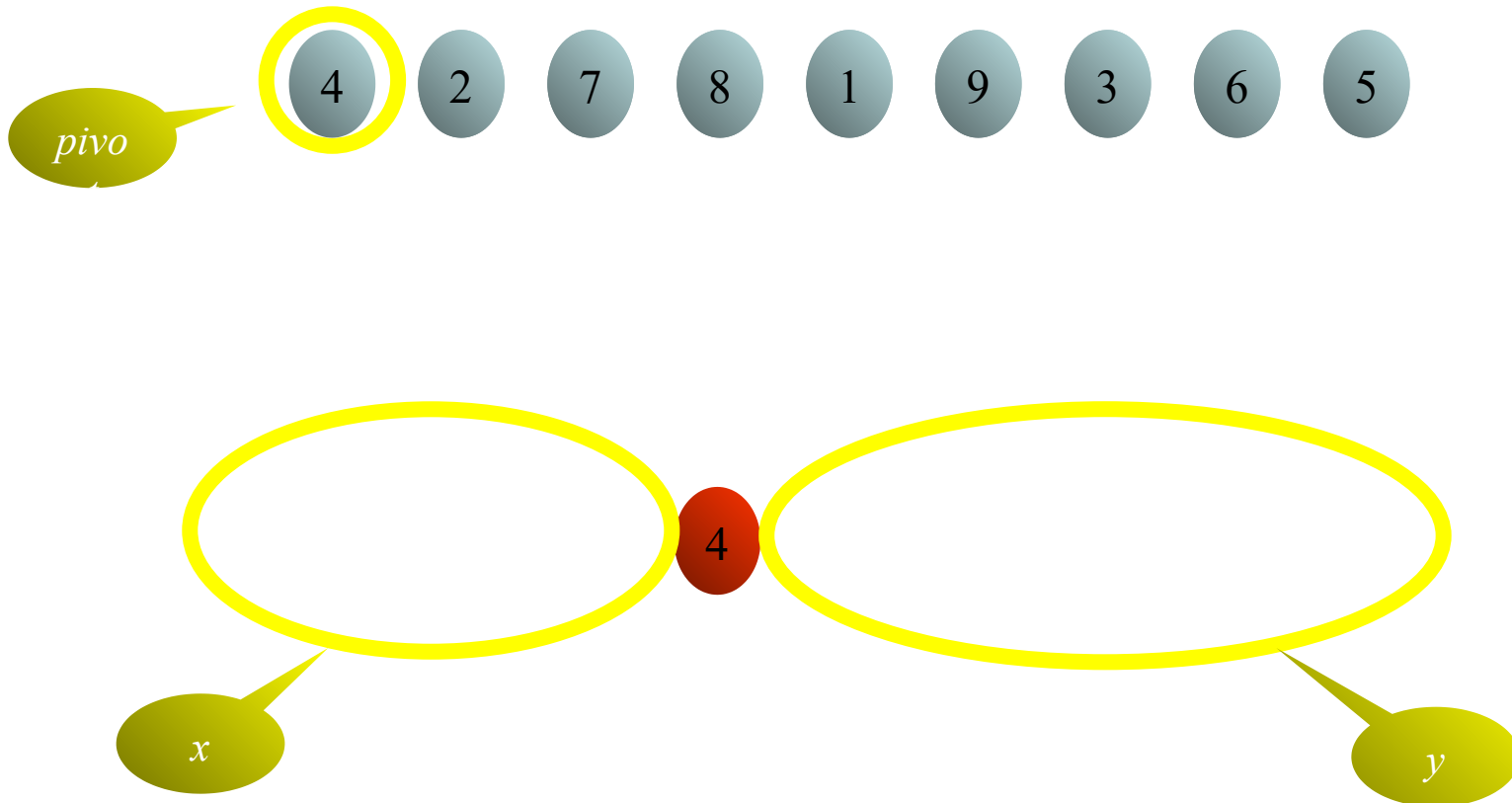
Summary

- Merge-Sort
 - Most of the work done in combining the solutions.
 - Best case takes $O(n \log(n))$ time
 - Average case takes $O(n \log(n))$ time
 - Worst case takes $O(n \log(n))$ time

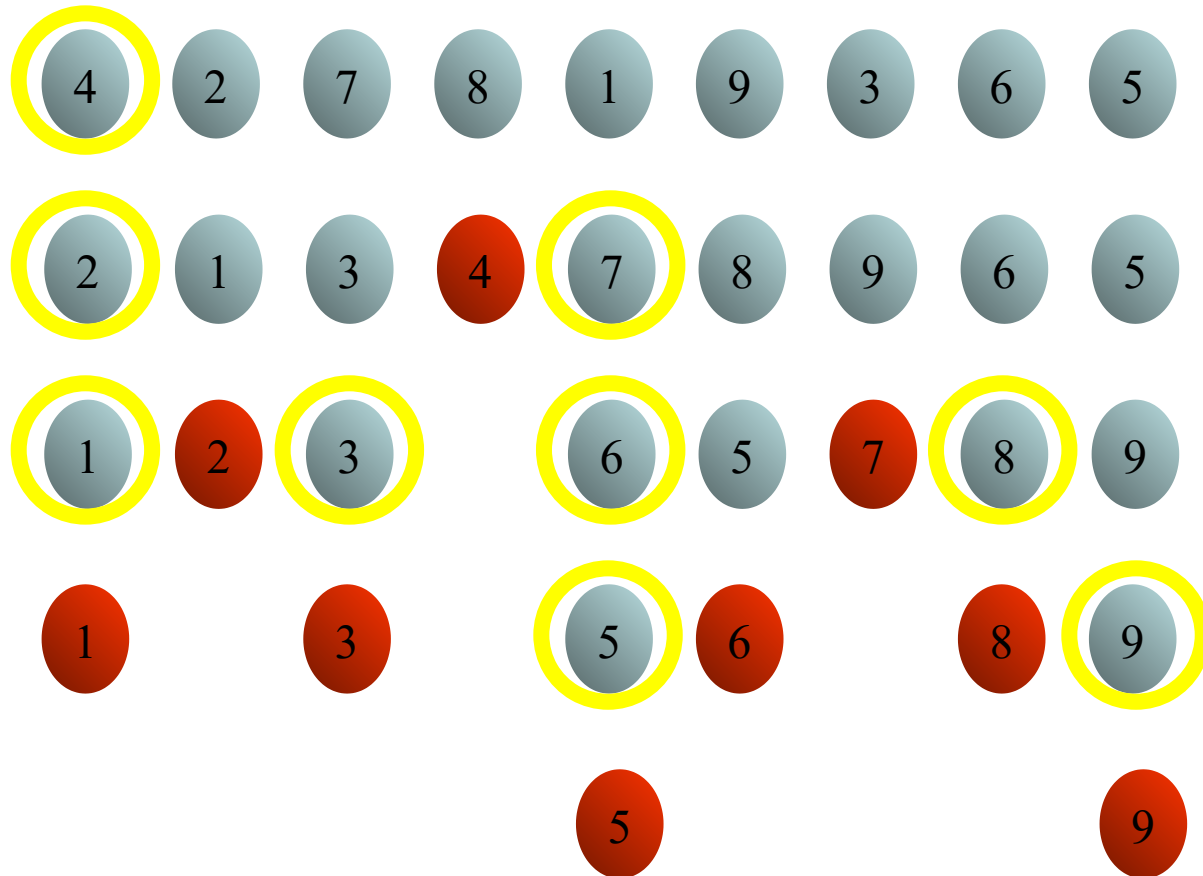
3. Quick Sort

- **Divide:**
 - Pick any element as the **pivot**, e.g, the first element
 - Partition the remaining elements into
 - FirstPart**, which contains all elements $< \text{pivot}$
 - SecondPart**, which contains all elements $> \text{pivot}$
- **Recursively sort** **FirstPart** and **SecondPart**.
- **Combine:** no work is necessary since sorting is done in place.

pivot divides *a* into two sublists *x* and *y*.



The whole process



Process:

Keep going from left side as long as $a[i] < \text{pivot}$ and from the right side as long as $a[j] > \text{pivot}$

pivot → 85 24 63 95 17 31 45 98

i

j

85 24 63 95 17 31 45 98

i

j

85 24 63 95 17 31 45 98

i

j

85 24 63 95 17 31 45 98

i

j

If $i < j$ interchange i^{th} and j^{th} elements and then Continue the process.

85	24	63	45	17	31	95	98
			i			j	

85	24	63	45	17	31	95	98
				i		j	

85	24	63	45	17	31	95	98
					i		

85 24 63 45 17 31 95 98

j

i

85 24 63 45 17 31 95 98

j

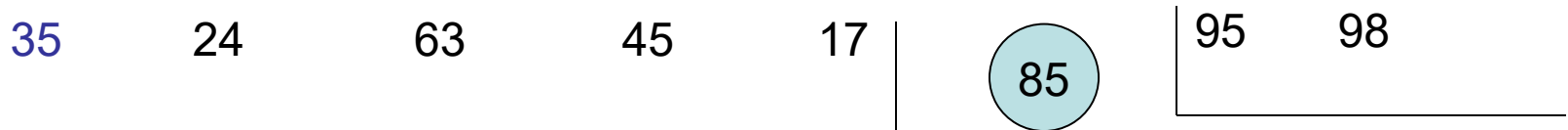
i

If $i \geq j$ interchange j^{th} and pivot elements
and then divide the list into two sublists.

35 24 63 45 17 85 95 98

j

Two sublists:



Recursively sort

SecondPart FirstPart and

QuickSort(low, j-1) QuickSort(j+1, high)

Quick Sort Algorithm :

Algorithm QuickSort(low,high)

//Sorts the elements $a[\text{low}], \dots, a[\text{high}]$ which resides

//in the global array $a[1:n]$ into ascending order;

// $a[n+1]$ is considered to be defined and must $\geq$ all the

// elements in $a[1:n]$.

{

if(low < high) *// if there are more than one element*

{ *// divide p into two subproblems.*

j := Partition(low,high);

// j is the position of the partitioning element.

QuickSort(low,j-1);

QuickSort(j+1,high);

// There is no need for combining solutions.

}

}

Algorithm Partition(low, high)

{

 pivot:= a[low]; i:=low; j:= high+1;

 while(i < j) do

 {

 i++;

 while(a[i] < pivot) do

 i++;

 j--;

 while(a[j] > pivot) do

 j--;

 if (i < j) then Interchange(i, j); // interchange i^{th}

and

 }

 // j^{th} elements.

 Interchange(low, j); return j; // interchange pivot and j^{th} element.

}

Algorithm interchange (x,y)

{

temp=a[x];

a[x]=a[y];

a[y]=temp;

}

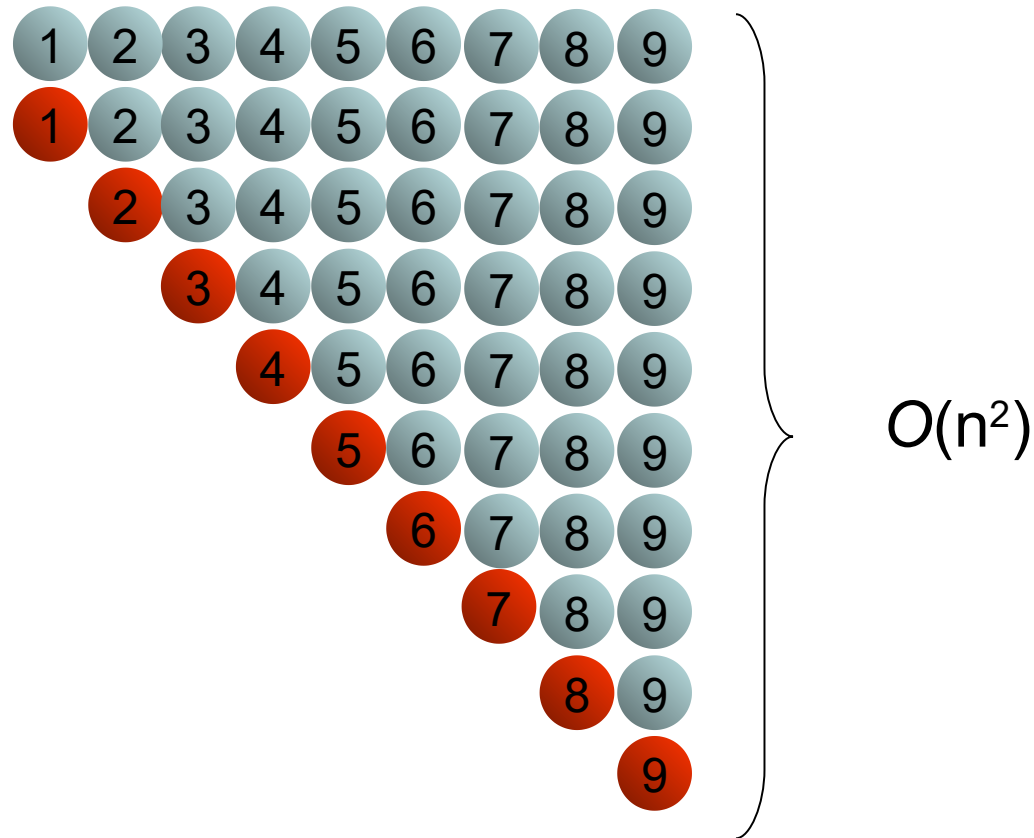
Time complexity analysis

- The time required to sort n elements using quicksort involves 3 components.
 - Time required for partitioning the array, which is *roughly proportional to n* .
 - Time required for sorting lower subarray.
 - Time required for sorting upper subarray.
- Assume that there are k elements in the lower subarray.
- Therefore,

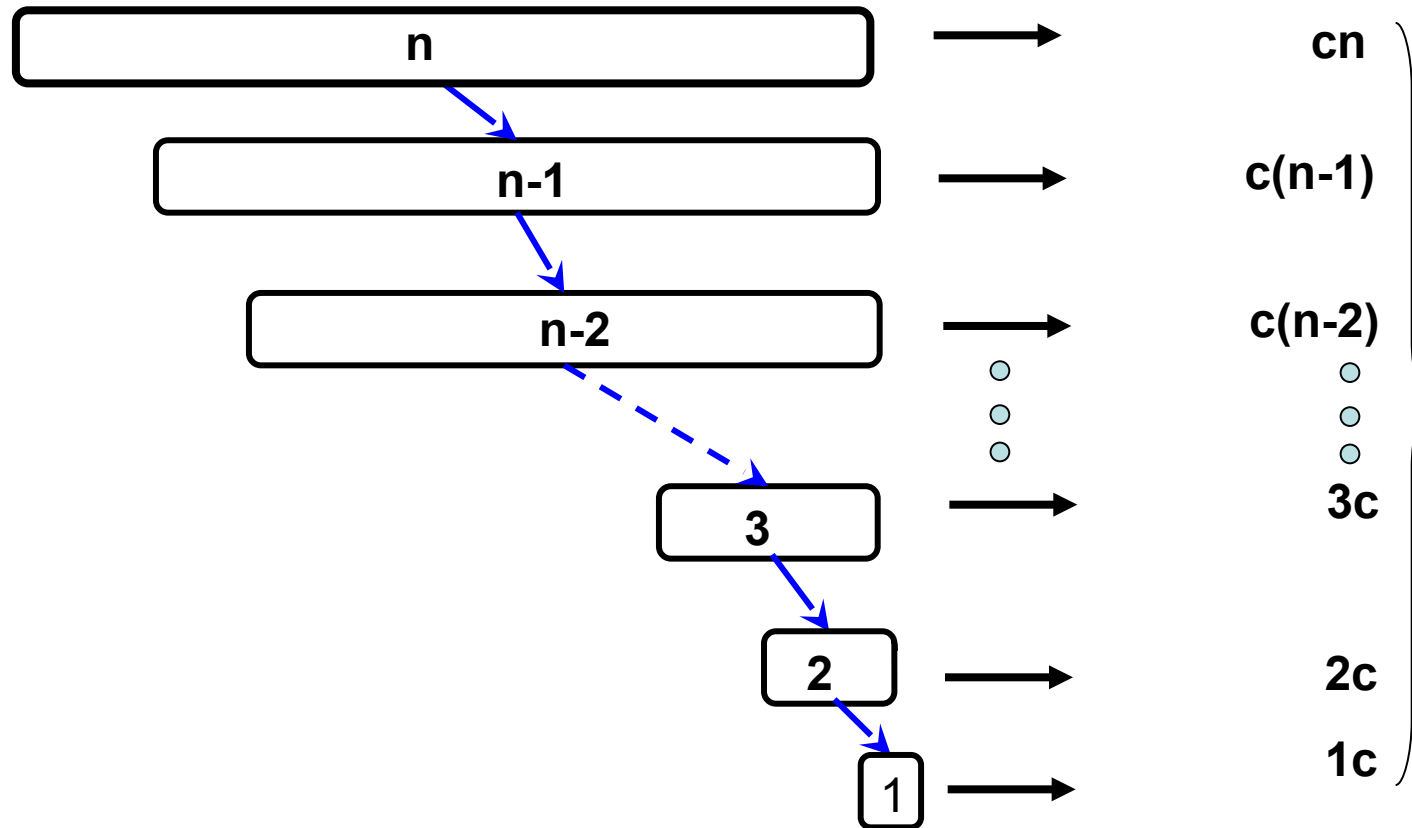
$$T(n) = \begin{cases} c_1 & n=1, c_1 \text{ is a constant} \\ T(k) + T(n-k-1) + c_2 n & n>1, c_2 \text{ is a constant} \end{cases}$$

A worst/bad case

It occurs if the list is already in sorted order



Worst/bad Case



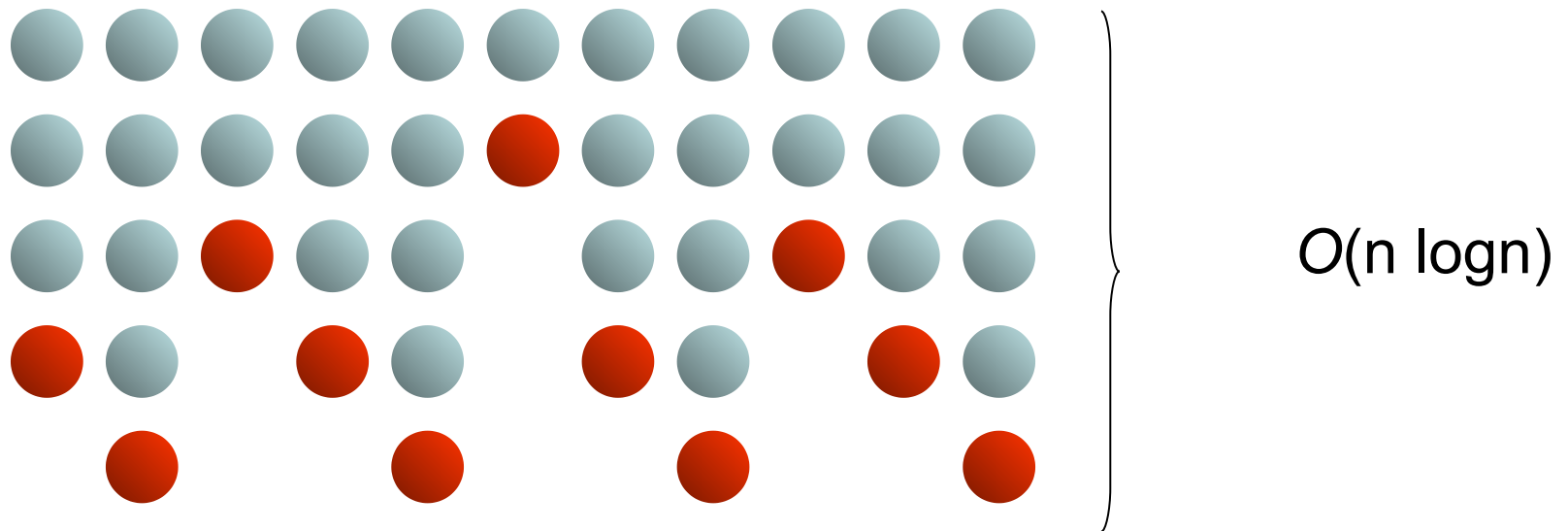
contd...

- In the worst case, the list is always partitioned into two sublists in which one of them is always empty. Thus , for the worst case analysis,

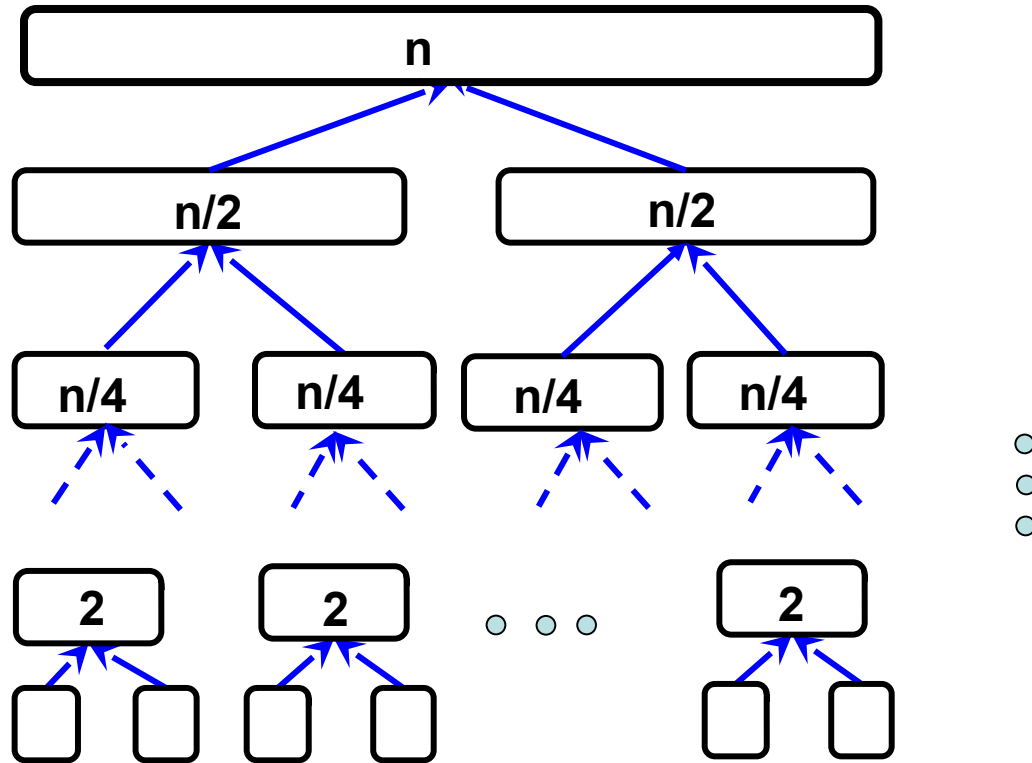
$$\begin{aligned} T(n) &= \begin{cases} T(n-1) + c_2 n & n > 1, c_2 \text{ is a constant} \end{cases} \\ &= T(n-1) + c_2 n \\ &= T(n-2) + c_2 (n-1) + c_2 n \\ &= T(n-3) + c_2 (n-2) + c_2 (n-1) + c_2 n \\ &\quad \dots \\ &\quad \dots \\ &= n(n+1)/2 = (n^2+n)/2 = \mathbf{O(n^2)} \end{aligned}$$

A Best/Good case

- It occurs only if each partition divides the list into two equal size sublists.



Best/Good Case



$$T(n) = \begin{cases} c_1 & n=1, c_1 \text{ is a constant} \\ 2T(n/2) + c_2 n & n>1, c_2 \text{ is a constant} \end{cases}$$

Assume $n=2^k$, then

$$\begin{aligned} T(n) &= 2T(n/2) + c_2 n \\ &= 2(2T(n/4) + c_2 n/2) + c_2 n \\ &= 4T(n/4) + 2c_2 n \\ &\dots\dots \\ &\dots\dots \\ &= 2^k T(1) + kc_2 n \\ &= c_1 n + c_2 n \log n = O(n \log n) \end{aligned}$$

Summary

- Quick-Sort
 - Most of the work done in partitioning
 - Best case takes $O(n \log(n))$ time
 - Average case takes $O(n \log(n))$ time
 - Worst case takes $O(n^2)$ time

4. Strassen's Matrix Multiplication

Basic Matrix Multiplication

Let A and B be two $n \times n$ matrices. The product $C=AB$ is also an $n \times n$ matrix.

```
void matrix_mult () {  
    for (i = 1; i <= n; i++) {  
  
        for (j = 1; j <= n; j++) {  
            for(k=1; k<=n; k++){  
                C[i,j]=C[i,j]+A[i,k]+B[k,j];  
            }  
        }  
    }  
}
```

Time complexity of above algorithm is
 $T(n)=O(n^3)$

Divide and Conquer Technique

- We want to compute the product $C=AB$, where each of A, B , and C are $n \times n$ matrices.
- Assume n is a power of 2.
- If n is not a power of 2, add enough rows and columns of zeros.
- We divide each of A, B , and C into four $n/2 \times n/2$ matrices, rewriting the equation $C=AB$ as follows:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} * \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Then,

$$\begin{aligned}
 C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\
 C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\
 C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\
 C_{22} &= A_{21}B_{12} + A_{22}B_{22}
 \end{aligned}
 \begin{bmatrix}
 \boxed{c_{11}} & \boxed{c_{12}} \\
 \boxed{c_{21}} & \boxed{c_{22}}
 \end{bmatrix}
 =
 \begin{bmatrix}
 \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} \\
 \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} \\
 \boxed{3} & \boxed{3} & \boxed{4} & \boxed{4} \\
 \boxed{3} & \boxed{3} & \boxed{4} & \boxed{4}
 \end{bmatrix}
 \times
 \begin{bmatrix}
 \boxed{5} & \boxed{5} & \boxed{6} & \boxed{6} \\
 \boxed{5} & \boxed{5} & \boxed{6} & \boxed{6} \\
 \boxed{7} & \boxed{7} & \boxed{8} & \boxed{8} \\
 \boxed{7} & \boxed{7} & \boxed{8} & \boxed{8}
 \end{bmatrix}$$

$\begin{matrix} A_{11} & A_{12} & & \\ & & B_{11} & B_{12} \\ & & & \\ A_{21} & A_{22} & B_{21} & B_{22} \end{matrix}$

- Each of these equations specifies two multiplications of size $n/2 \times n/2$ and the addition of their $n/2 \times n/2$ products.
- We can derive the following recurrence relation for the time $T(n)$ to multiply two $n \times n$ matrices:

$$T(n) = \begin{cases} c_1 & \text{if } n \leq 2 \\ 8T(n/2) + c_2n^2 & \text{if } n > 2 \end{cases}$$

$$T(n) = O(n^3)$$

- This method is no faster than the ordinary method.

$$T(n) = 8T(n/2) + c_2 n^2$$

$$= 8 \left[8T(n/4) + c_2 (n/2)^2 \right] + c_2 n^2$$

$$= 8^2 T(n/4) + c_2 2n^2 + c_2 n^2$$

$$= 8^2 \left[8T(n/8) + c_2 (n/4)^2 \right] + c_2 2n^2 + c_2 n^2$$

$$= 8^3 T(n/8) + c_2 4n^2 + c_2 2n^2 + c_2 n^2$$

:

$$= 8^k T(1) + \dots + c_2 4n^2 + c_2 2n^2 + c_2 n^2$$

$$= 8^{\log_2 n} c_1 + c_2 n^2 .$$

$$= n^{\log_2 8} c_1 + c_2 n^2 = n^3 c_1 + c_2 n^2 = O(n^3)$$

Strassen's method

- Matrix multiplications are more expensive than matrix additions or subtractions($O(n^3)$ versus $O(n^2)$).
- Strassen has discovered a way to compute the multiplication using only 7 multiplications and 18 additions or subtractions.
- His method involves computing 7 $n/2 \times n/2$ matrices $M_1, M_2, M_3, M_4, M_5, M_6$, and M_7 , then c_{ij} 's are calculated using these matrices.

Algorithm 3 Strassen's Matrix Multiplication Algorithm

Input: $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$ and $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \in \mathbb{R}^{n \times n}$

```
1:  if  $n = 1$  then
2:     $C = A \cdot B$ 
3:  else
4:     $M_1 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$ 
5:     $M_2 = (A_{21} + A_{22}) \cdot B_{11}$ 
6:     $M_3 = A_{11} \cdot (B_{12} - B_{22})$ 
7:     $M_4 = A_{22} \cdot (B_{21} - B_{11})$ 
8:     $M_5 = (A_{11} + A_{12}) \cdot B_{22}$ 
9:     $M_6 = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$ 
10:    $M_7 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$ 
11:    $C_{11} = M_1 + M_4 - M_5 + M_7$ 
12:    $C_{12} = M_3 + M_5$ 
13:    $C_{21} = M_2 + M_4$ 
14:    $C_{22} = M_1 - M_2 + M_3 + M_6$ 
```

Output: $A \cdot B = C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \in \mathbb{R}^{n \times n}$

Formulas for Strassen's Algorithm

$$M_1 = (A_{11} + A_{22}) * (B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22}) * B_{11}$$

$$M_3 = A_{11} * (B_{12} - B_{22})$$

$$M_4 = A_{22} * (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12}) * B_{22}$$

$$M_6 = (A_{21} - A_{11}) * (B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22}) * (B_{21} + B_{22})$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 + M_3 - M_2 + M_6$$

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} * \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$= \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

The resulting recurrence relation for T(n) is

$$T(n) = \begin{cases} c_1 & n \leq 2 \\ 7T(n/2) + c_2 n^2 & n > 2 \end{cases}$$

$$T(n) = 7^k T(1) + c_2 n^{2-1} \left[\frac{7}{4} + \left(\frac{7}{4}\right)^2 + \left(\frac{7}{4}\right)^3 + \dots + \left(\frac{7}{4}\right)^{k-1} \right]$$

$$\therefore S_n = a + ar + ar^2 + \dots + ar^{n-1}$$

$$\text{When } r > 1, S_n = a \frac{(r^n - 1)}{(r - 1)}$$

$$= 7^{\log_2 n} c_1 + c_2 n^2 \left(\frac{7}{4}\right)^{\log_2 n}$$

$$= c_1 n^{\log_2 7} + c_2 n^{\log_2 4} (n^{\log_2 7 - \log_2 4})$$

$$= c_1 n^{\log_2 7} + c_2 (n^{\log_2 4 + \log_2 7 - \log_2 4}) = c_1 n^{\log_2 7} + c_2 n^{\log_2 7}$$

$$= c n^{\log_2 7} = O(n^{\log_2 7}) \sim O(n^{2.81})$$